TransferLog

App de gestión de turnos y vacaciones

MEMORIA DE PROYECTO



CICLO: Desarrollo de Aplicaciones Multiplataforma (DAM)
CENTRO: CIFP César Manrique
ALUMNOS: Sergio Fernández, Gisela Estévez Moleiro
TUTOR: Juan Carlos
CURSO: 2025 / 2026
FECHA: Abril 2026
PERIODO DE DOCUMENTACIÓN: 30/03/2026 – 25/04/2026

1. Índice

1. Índice **2**
2. Introducción **5**
3. Origen, contextualización y justificación **6**
4. Objetivo del proyecto **7**
4.1 Objetivos específicos 7
5. Descripción del proyecto **8**
5.1 Perfil Empleado 8
5.2 Perfil Administrador 8
5.3 Stack tecnológico 8
5.4 Estructura del proyecto 9
5.5 Cambios respecto al anteproyecto 9
6. Tareas efectivas realizadas, esfuerzo y problemas encontrados **10**
6.1 Criterio utilizado para documentar el trabajo 10
6.2 Resumen global del esfuerzo ajustado 11
6.3 Cronología resumida de ejecución 11
6.3 Ajuste de tiempos por tareas principales 12
6.4 Problemas principales encontrados y soluciones aplicadas 13
6.5 Riesgos y mejoras futuras identificadas 14
6.6 Conclusión 14
7. Bibliografía y referencias **14**
7.1 Documentación oficial 14
7.2 Recursos de apoyo 14
8. Anexos **15**
Anexo I – Modelo entidad-relación / Modelo relacional 15
Anexo II – Diagramas de clases 16
1. Capa de servicios (src/database/) 16
2. Capa de contexto (src/context/) 21
3. Capa de pantallas (src/screens/) 22
4. Capa de pantallas (src/screens/Admin) 25
Anexo III – Casos de uso 27
Actor: Empleado 28
Actor: Administrador 30
Anexo IV – Capturas de pantalla de la aplicación 34
Anexo V – Testing del sistema 42
Anexo VI – Instrucciones de instalación y ejecución 45

2. Introducción

La presente memoria recoge el proceso de desarrollo completo del proyecto TransferLog, una aplicación móvil multiplataforma orientada a la gestión de turnos laborales y vacaciones en empresas de transporte de pasajeros (transfer).

El proyecto ha sido desarrollado por el Grupo 3 del Módulo de Proyecto del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM), durante el curso 2025/2026. A lo largo de este documento se describe el origen del problema que motivó la aplicación, los objetivos planteados, la descripción técnica de la solución implementada, las tareas realizadas con sus tiempos reales, las dificultades encontradas durante el desarrollo, y los anexos técnicos con los modelos de datos, diagramas de clases y casos de uso.

TransferLog ha evolucionado respecto al planteamiento inicial del anteproyecto, adoptando una arquitectura en la nube mediante Supabase (PostgreSQL) en lugar del almacenamiento local SQLite originalmente previsto. Esta decisión técnica fue tomada durante la fase de diseño para permitir la sincronización en tiempo real entre dispositivos y una gestión centralizada de datos más robusta y escalable.

3. Origen, contextualización y justificación

Las empresas de transporte de pasajeros tipo transfer trabajan con turnos rotativos y horarios de mañana, tarde y noche, ya que deben ofrecer servicio en todo momento y coordinarse constantemente entre conductores y personal de oficina.

La observación directa del funcionamiento de estas empresas reveló que la gran mayoría gestiona sus recursos humanos mediante hojas de cálculo compartidas, correos electrónicos y comunicaciones informales. Esta práctica genera problemas recurrentes:

- Errores frecuentes en la asignación de turnos, especialmente en periodos de vacaciones solapadas.
- Falta de visibilidad del estado de las solicitudes de vacaciones por parte del empleado.
- Dificultad para coordinar cambios de turno de manera ágil y trazable.
- Ausencia de un historial fiable de turnos realizados por cada trabajador.
- Imposibilidad de generar informes de horas por tipo de jornada sin trabajo manual intensivo.

TransferLog surge como respuesta directa a este diagnóstico, proponiendo una herramienta digital centralizada que permita a administradores y empleados interactuar con la planificación laboral desde sus dispositivos móviles, en tiempo real y con independencia de la ubicación física.

El proyecto se enmarca dentro de los contenidos del Módulo de Proyecto del CFGS DAM, aplicando de manera integrada competencias adquiridas en módulos como Programación Multimedia y Dispositivos Móviles, Acceso a Datos, Sistemas de Gestión Empresarial y Programación de Servicios y Procesos.

4. Objetivo del proyecto

El objetivo principal del proyecto es desarrollar una aplicación móvil multiplataforma que permita a las empresas de transporte de pasajeros gestionar de forma digital, centralizada y eficiente los turnos laborales de su plantilla y el proceso de solicitud y aprobación de vacaciones.

4.1 Objetivos específicos

- Diseñar e implementar un sistema de autenticación con diferenciación de roles (Empleado / Administrador).
- Desarrollar una vista de calendario interactivo que muestre turnos y vacaciones de forma visual e intuitiva.
- Implementar un flujo completo de solicitud, aprobación y cancelación de vacaciones con cálculo automático de días disponibles.
- Crear un sistema de asignación de turnos por parte del administrador con validación de conflictos.
- Ofrecer un módulo de reportes que compute automáticamente las horas y turnos realizados por cada empleado en un período mensual.
- Garantizar la persistencia y sincronización de datos en la nube mediante Supabase (PostgreSQL).
- Desarrollar la aplicación con React Native y Expo para asegurar compatibilidad multiplataforma (iOS y Android).

5. Descripción del proyecto

TransferLog es una aplicación móvil desarrollada con React Native (Expo SDK 50+) que conecta con un backend en la nube mediante Supabase, el cual provee una base de datos PostgreSQL gestionada, autenticación de usuarios y API REST/Realtime automática.

La aplicación presenta dos experiencias de usuario diferenciadas según el rol:

5.1 Perfil Empleado

- Consulta de su propio calendario de turnos en vista semanal y mensual.
- Visualización de vacaciones pendientes y aprobadas directamente en el calendario.
- Solicitud de vacaciones con selector de fechas, cálculo de días disponibles y campo de motivo opcional.
- Cambio de contraseña con validación de identidad.

5.2 Perfil Administrador

- Gestión completa de empleados: alta, baja y edición.
- Asignación de turnos con validación de conflictos (no permite asignar turno en fechas de vacaciones aprobadas ni en días con turno ya asignado).
- Herramienta de copia rápida de turnos de la semana anterior (operación transaccional).
- Gestión de solicitudes de vacaciones: aprobada, pendiente y rechazada.
- Acceso al módulo de reportes mensuales: total de turnos por empleado, desglose por tipo (Mañana, Tarde, Noche), días de vacaciones disfrutados y exportar el reporte a PDF.

5.3 Stack tecnológico

Capa	Tecnología	Versión / Detalle
Frontend móvil	React Native + Expo	Expo SDK 50+
Navegación	React Navigation	Stack & Bottom Tabs
Backend / BD	Supabase (PostgreSQL)	Proyecto en la nube
Autenticación	Supabase Auth	Email + password
Gestión de fechas	date-fns	Cálculos y zonas horarias
Iconografía	Material Community Icons	React Native Vector Icons
IDE / Entorno	VS Code + Expo Go	Desarrollo y pruebas

5.4 Estructura del proyecto

El proyecto sigue una arquitectura de carpetas modular orientada a la separación de responsabilidades:

- **src/components/** – Componentes de UI reutilizables (ShiftBadge, VacationCard, etc.)
- **src/context/** – Estado global de la sesión mediante AuthContext.
- **src/database/** – Servicios de acceso a datos (attendanceService, employeeService, shiftService, vacationService y workCenterService).
- **src/lib/** – Conexión de Supabase, fecha y horas, crear PDFs y Ubicación / GPS.
- **src/navigation/** – Definición de rutas, stacks y pestañas por rol.
- **src/screens/** – Pantallas de la aplicación (Home, Calendar, Vacations, Admin, Reports…).
- **src/theme/** – Sistema de diseño: colores, tipografía y constantes de estilo.
- **src/config/** – Configuración del calendario en español y variables de entorno.

5.5 Cambios respecto al anteproyecto

Durante el desarrollo se tomaron las siguientes decisiones de cambio respecto al anteproyecto original:

Elemento	Anteproyecto	Implementación final
Almacenamiento	SQLite local	Supabase (PostgreSQL en la nube)
Sincronización	Sin sincronización	Tiempo real entre dispositivos
Autenticación	No contemplada formalmente	Supabase Auth con roles en BD
Reportes	No incluido	Módulo de estadísticas mensuales
Copia de turnos	No incluido	Herramienta transaccional de copia semanal
IDE previsto	Android Studio	VS Code + Expo Go

6. Tareas efectivas realizadas, esfuerzo y problemas encontrados

A continuación, se detallan las tareas realizadas a partir del historial de commits, ***issues_backlog.md***, auditorías técnicas, migraciones de Supabase, Edge Functions, pruebas automatizadas y la estructura actual del proyecto, junto con el tiempo empleado y las dificultades encontradas..

6.1 Criterio utilizado para documentar el trabajo

Este documento resume las tareas efectivamente realizadas durante el desarrollo, estabilización y documentación del proyecto **TransferLog**.

La fecha de inicio se ha fijado en el **30/03/2026**, tomando como referencia el primer commit registrado en el repositorio:

daeb6ac 2026-03-30 17:42:15 Initial commit

La fecha final de referencia es el **25/04/2026**, momento en el que el proyecto se encuentra en una fase avanzada de desarrollo, con las funcionalidades principales implementadas, refactorizaciones relevantes aplicadas, pruebas automatizadas incorporadas y documentación técnica generada.

Los tiempos indicados son **estimaciones razonadas** de dedicación efectiva. Se han calculado considerando una dedicación media aproximada de entre 2 y 3 horas diarias durante los 27 días naturales del periodo:

- **Esfuerzo mínimo:** 27 días × 2 h/día = 54 h
- **Esfuerzo máximo:** 27 días × 3 h/día = 81 h
- **Valor medio de referencia:** 27 días × 2,5 h/día = **67,5 h**

Por ello, el esfuerzo total documentado se estima en **68 horas efectivas** de trabajo, distribuidas entre análisis, diseño, implementación, pruebas, integración con Supabase, refactorización y documentación.

6.2 Resumen global del esfuerzo ajustado

Bloque de trabajo	Tareas principales	Esfuerzo estimado
Seguridad y Autenticación	Migración a Supabase Auth, RLS, cambio forzado de contraseña y limpieza.	12 h
Vacaciones e Integridad	Corrección de fechas, aprobación/ cancelación transaccional y saldo de días.	10 h
Fichajes y Control Horario	Monitor admin, geolocalización, anulación auditada y políticas por empleado.	13 h
Panel de Administración	Refactor estructural, carga modular, gestión de empleados y turnos.	11 h
Reportes, PDF y UI	Exportación mensual, restauración UI premium, dashboard y navegación.	8 h
Testing y Documentación	Jest, mocks Supabase, auditorías técnicas y mapa documental.	8 h
Limpieza y Dependencias	Eliminación SQLite, hooks, .gitignore y configuración EAS.	6 h
Total Estimado		68 h

6.3 Cronología resumida de ejecución

Fecha	Trabajo realizado	Evidencia
30/03/2026	Inicio del repositorio y primer commit.	Commit Initial commit
31/03/2026	Inicialización Expo, estructura base, autenticación local y SQLite.	Commits inicialización
01/04/2026	Incorporación del panel de administración y logo.	Commits panel admin
14/04/2026	Exportación PDF, correcciones JSX y monitor premium de fichajes.	Commits feat(pdf), fix(ui)
15/04/2026	Limpieza SQLite, auditoría inicial y backlog técnico.	Commits #19, #23
16/04/2026	Migración a Supabase Auth, RLS y provisionado seguro.	Migración 20260416...sql
17/04/2026	Corrección de fechas en edición admin de vacaciones.	Commit fix(vacaciones)
18/04/2026	Vacaciones transaccionales y geolocalización de fichaje.	Migraciones #30, #33, #34
19/04/2026	Anulación auditada y auditoría profunda.	Migraciones #35, #26, #37
20/04/2026	Refactor de AdminScreen y corrección RLS contraseña.	Commits #40, #44

Fecha	Trabajo realizado	Evidencia
30/03/2026	Inicio del repositorio y primer commit.	Commit Initial commit
21/04/2026	Baja integral de empleados y compatibilidad Edge Function.	Commits #42, #43
22/04/2026	Testing Jest, auditoría complementaria y configuración EAS.	Commits de test
24/04/2026	Documentación final y generación de mapa documental.	Commit Documentacion
25/04/2026	Revisión final de memoria y consolidación documental.	Memoria final

6.3 Ajuste de tiempos por tareas principales

Seguridad y autenticación

- Migración a Supabase Auth: **5 h**
- Corrección RLS en primer cambio de contraseña: **2 h**
- Seguridad, .env, Husky y limpieza de credenciales: **2 h**
- Revisión y ajustes de políticas RLS: **3 h**
- **Subtotal: 12 h**

Vacaciones, fechas e integridad de datos

- Corrección de fechas en edición admin: **2 h**
- Aprobación y cancelación transaccional: **3 h**
- Edición transaccional de vacaciones: **2 h**
- Cancelación de vacaciones aprobadas y selector táctil: **3 h**
- **Subtotal: 10 h**

Fichajes y control horario

- Registro básico de jornada: **2 h**
- Monitor de fichajes en panel admin (Diseño y funcional): **4 h**
- Geolocalización flexible por empleado: **4 h**
- Anulación auditada y registro manual por admin: **3 h**
- **Subtotal: 13 h**

Empleados, perfil y administración

- Gestión de empleados y baja integral: **4 h**
- Foto de perfil por usuario: **2 h**
- Refactor estructural de AdminScreen.js y carga modular: **4 h**
- Correcciones de inconsistencias A-07/A-08: **1 h**
- **Subtotal: 11 h**

Turnos, calendario y reportes

- Asignación masiva y plantillas horarias: **3 h**
- Validación de conflictos y copia semanal: **2 h**
- Reportes mensuales y exportación PDF: **3 h**
- **Subtotal: 8 h**

Migración de infraestructura y limpieza técnica

- Migración de SQLite a Supabase Cloud: **3 h**
- Limpieza de código y consolidación: **1 h**
- Actualización de dependencias y configuración EAS/Hooks: **2 h**
- **Subtotal: 6 h**

Testing y calidad

- Configuración de Jest, jest-expo y mocks: **3 h**
- Tests de vacationService: **2 h**
- Auditorías técnicas y documentación final: **3 h**
- **Subtotal: 8 h**

6.4 Problemas principales encontrados y soluciones aplicadas

Durante el desarrollo se detectaron desafíos técnicos que obligaron a replantear la arquitectura:

1. **Credenciales inseguras:** Se solucionó migrando a **Supabase Auth**, eliminando la gestión de contraseñas en el cliente.
2. **Operaciones no transaccionales:** Las vacaciones afectaban al saldo antes de confirmarse. Se implementaron **RPCs (Remote Procedure Calls)** en la base de datos para asegurar la atomicidad.
3. **Bloqueos por RLS:** Las políticas de seguridad impedían al usuario cambiar su propia contraseña en el primer inicio. Se ajustaron las políticas para permitir el acceso al campo password_changed.
4. **Complejidad en AdminScreen:** El archivo superaba las 1000 líneas. Se realizó un **refactor modular** separando componentes (Empleados, Turnos, Fichajes).
5. **Fragilidad en geolocalización:** Se detectaron errores por falta de permisos. Se implementó una lógica de **geolocalización flexible** que permite fichar sin coordenadas si el perfil del empleado lo autoriza.
6. **Incompatibilidad de Testing:** Conflictos entre Jest y el SDK de Expo. Se solucionó configurando un **mock global** para el cliente de Supabase.

6.5 Riesgos y mejoras futuras identificadas

- **Validación en Backend:** Trasladar la validación de secuencias entrada/salida a *triggers* de base de datos.
- **Cobertura de Tests:** Ampliar las pruebas unitarias a attendanceService y Edge Functions.
- **Normalización de Fechas:** Sustituir el uso de toISOString() por una librería de manejo de zonas horarias para evitar desfases en el cálculo de "hoy".
- **Accesibilidad:** Incorporar etiquetas ARIA y roles de accesibilidad en los controles táctiles.

6.6 Conclusión

El proyecto **TransferLog** ha evolucionado de un prototipo con persistencia local a una aplicación robusta basada en la nube. El esfuerzo de **68 horas** refleja no solo la creación de interfaces, sino un trabajo profundo en la integridad de los datos y la seguridad.

El aprendizaje clave ha sido la importancia de delegar las reglas críticas de negocio al backend (Supabase/PostgreSQL) para garantizar que, independientemente del estado del cliente móvil, la información permanezca consistente y protegida.

7. Bibliografía y referencias

A continuación se listan las fuentes consultadas durante el desarrollo del proyecto:

7.1 Documentación oficial

- React Native Documentation
- Expo Documentation
- Supabase Documentation
- React Navigation Documentation
- date-fns Documentation

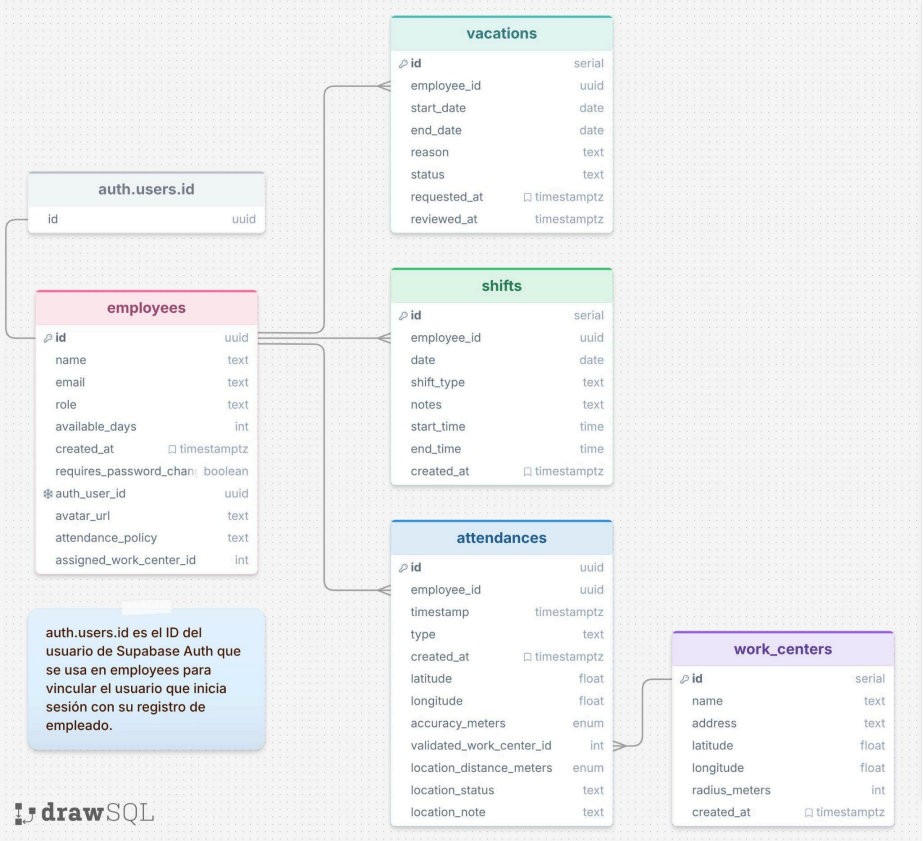
7.2 Recursos de apoyo

- Stack Overflow – Resolución de problemas técnicos específicos.
- Supabase GitHub Issues – Consulta de comportamientos conocidos de la plataforma.
- Material Community Icons – Catálogo de iconos utilizados.
- Anthropic Claude (IA generativa) – Apoyo en generación de código, revisión de lógica y documentación.

8. Anexos

Anexo I – Modelo entidad-relación / Modelo relacional

El modelo de datos de TransferLog está alojado en Supabase (PostgreSQL). A continuación, se muestra el diagrama relacional.



Descripción de tablas principales

La base de datos consta de las siguientes entidades principales:

Tabla	Descripción	Relaciones principales
employees	Almacena los datos de cada empleado (nombre, email, rol, días de vacaciones disponibles).	1:N con shifts, vacations y attendances. N:1 con work_centers 1:1 con auth.users
shifts	Registra los turnos asignados a cada empleado. Cada fila representa un turno en una fecha concreta.	N:1 con employees
vacations	Gestiona las solicitudes de vacaciones: fechas, estado (pending/approved/rejected) y motivo.	N:1 con employees
attendances	Registro de asistencia (entrada/salida, ubicación, validación de centro de trabajo)	N:1 con employees y work_centers
work_centers	Centros de trabajo con ubicación geográfica y radio permitido	1:N con employees y attendances

Anexo II – Diagramas de clases

Los diagramas de clases reflejan la arquitectura del código fuente de la aplicación, organizada en capas de servicios (database), contexto (context) y pantallas (screens).

1. Capa de servicios (src/database/)

Esta es la sección del proyecto donde se agrupa todo el código encargado de **manejar los datos de la base de datos de supabase y aplicar la lógica de funcionamiento de la aplicación**.

attendanceService.js

Este sistema gestiona los fichajes de empleados.

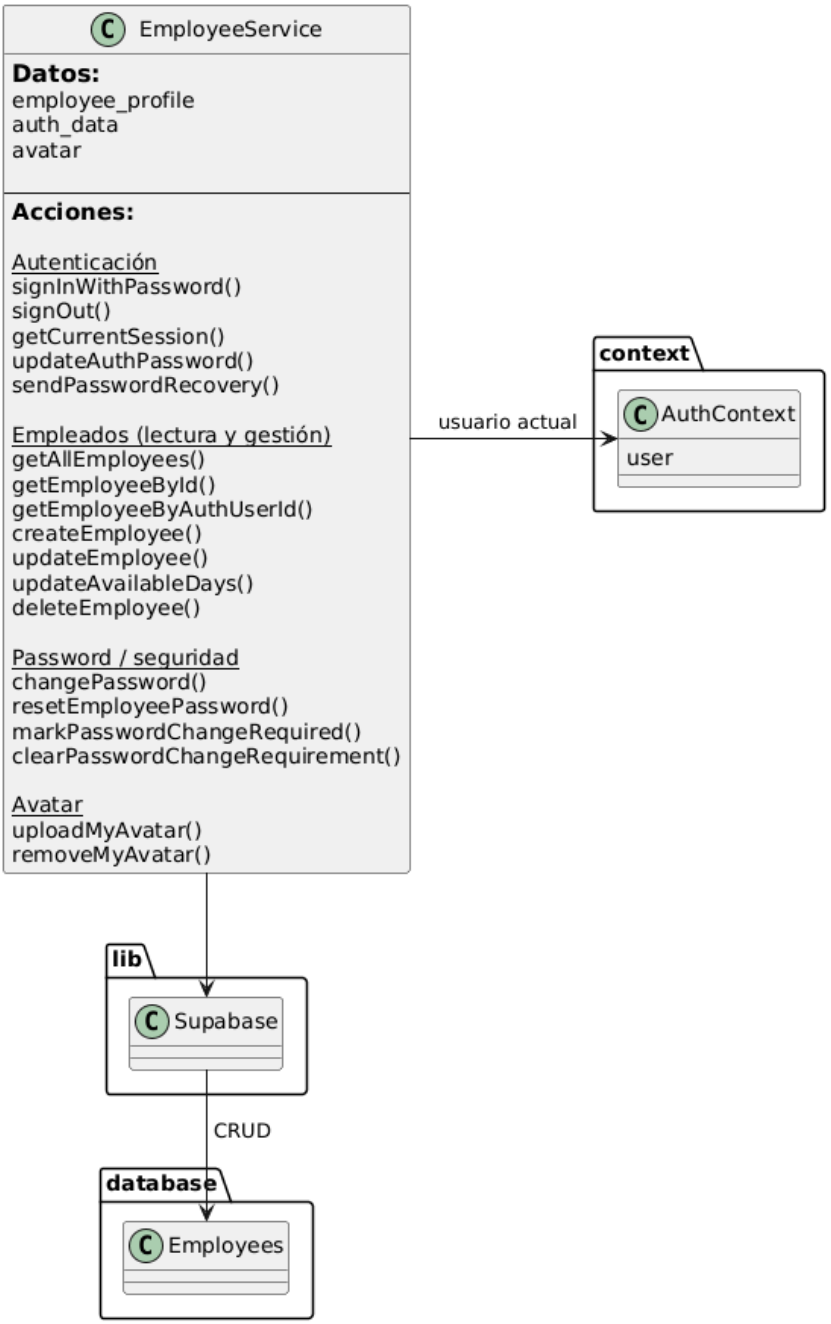
- Registra entradas y salidas evitando duplicados.
- Permite consultar y organizar los fichajes.
- Valida la ubicación según la política del empleado.
- Permite a administradores crear y anular fichajes con reglas de control.



employeeService.js

Se encarga de la gestión general de usuarios dentro del sistema.

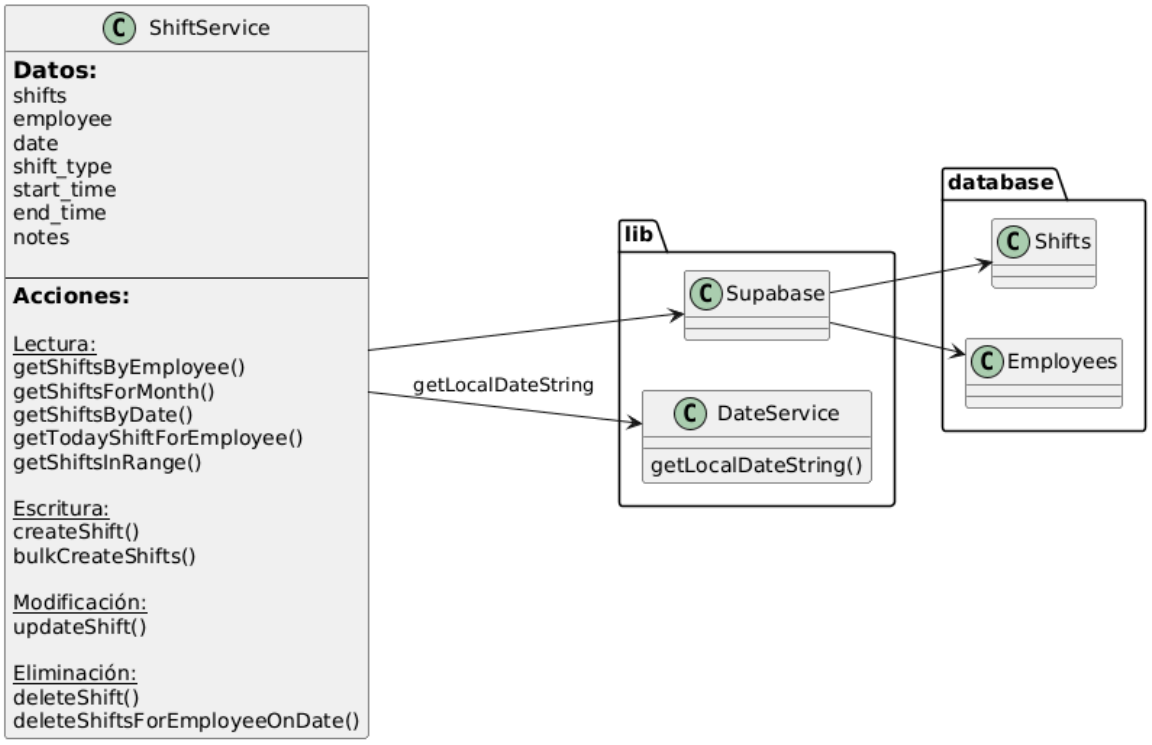
- Gestiona empleados y la autenticación del sistema (login y sesión).
- Permite iniciar y cerrar sesión, y obtener datos o perfiles de empleados.
- Incluye funciones para crear, actualizar y eliminar empleados, además de gestionar contraseñas y restablecimientos.
- Maneja la subida y eliminación de avatares de los usuarios.



shiftService.js

Se encarga de gestionar los **turnos** de los empleados.

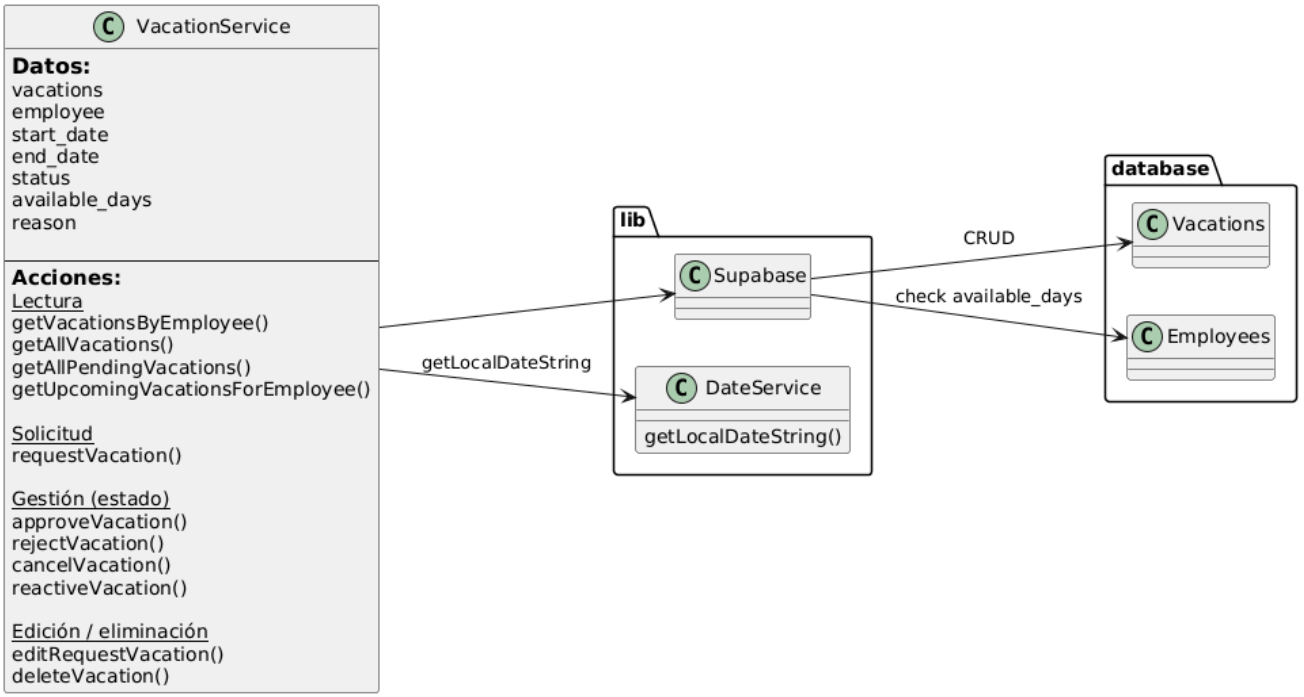
- Usa plantillas de turnos (mañana, tarde y noche).
- Permite consultar turnos por empleado, fecha o rango de fechas.
- Incluye gestión completa: crear, actualizar y eliminar turnos.
- Soporta la creación de turnos.
- Si no se indican horarios, aplica automáticamente la plantilla del turno.



vacationService.js

Se encarga de gestionar las vacaciones de los empleados.

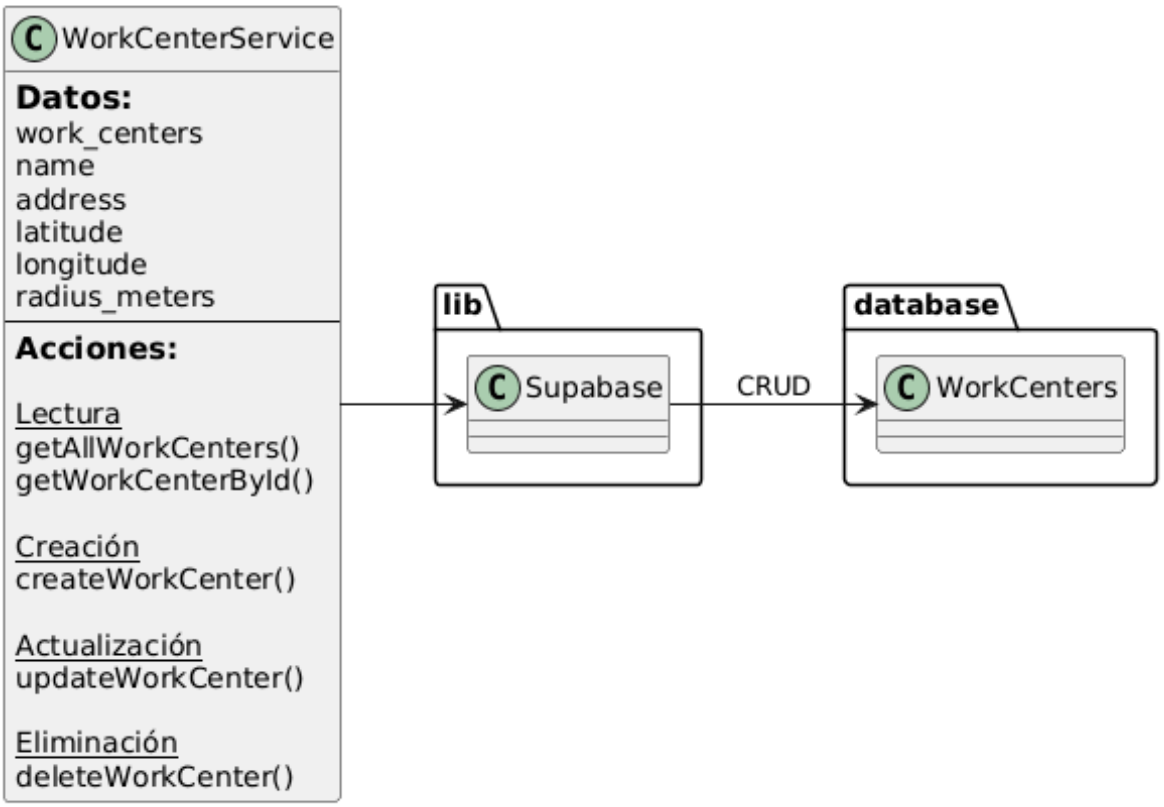
- Permite consultar vacaciones por empleado, estado o próximas.
- Calcula los días solicitados y valida disponibilidad.
- Permite crear solicitudes con fechas de inicio y fin.
- Incluye gestión de aprobación, rechazo, edición y cancelación.
- También permite reactivar o eliminar solicitudes.



workCenterService.js

Se encarga de gestionar los centros de trabajo.

- Permite consultar todos los centros o uno por ID.
- Incluye creación, actualización y eliminación de centros.
- Guarda datos como nombre, ubicación y radio permitido.
- Define los lugares donde los empleados pueden fichar.

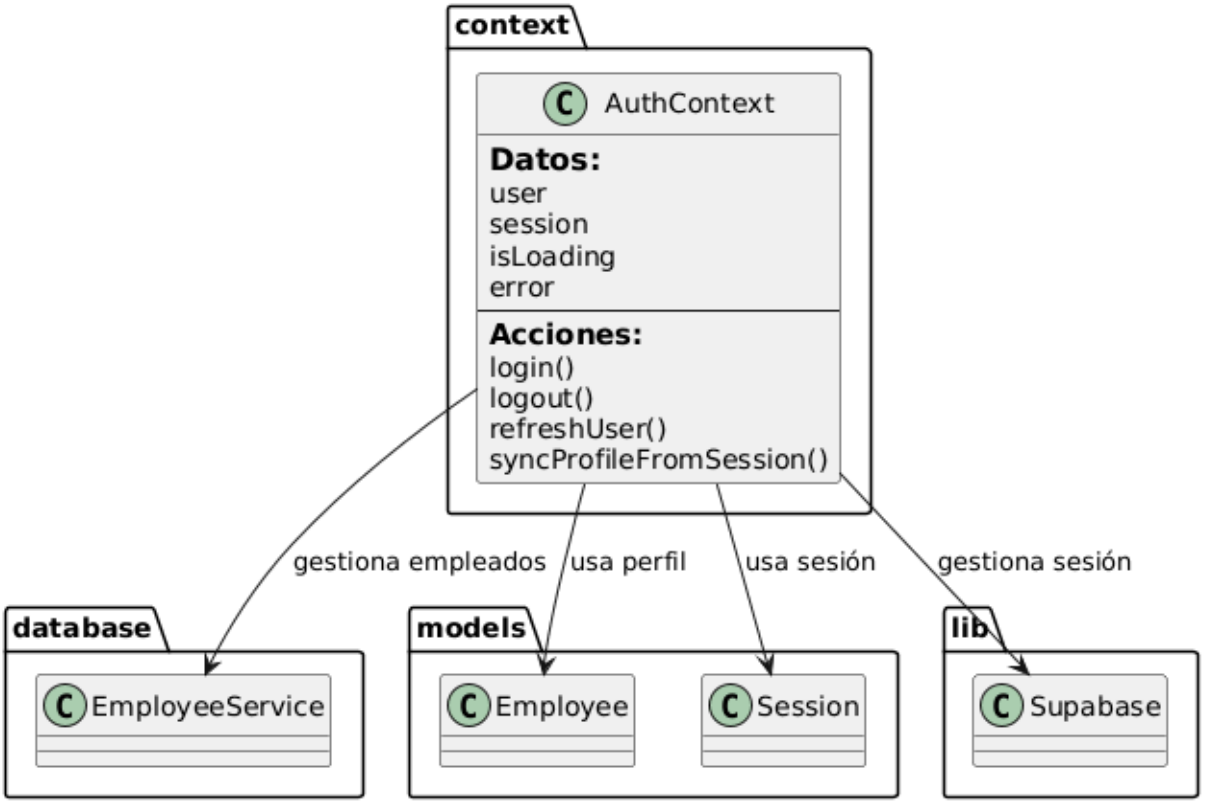


2. Capa de contexto (src/context/)

providerContext.js

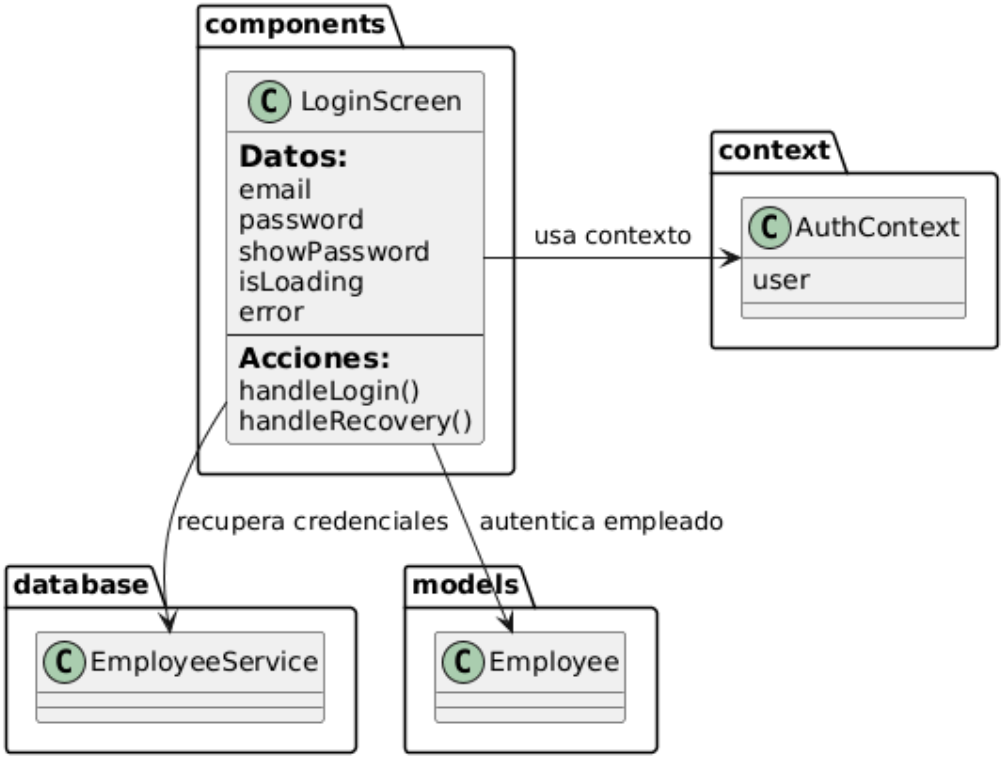
Este código gestiona la autenticación de la aplicación.

- Controla el estado de sesión, usuario, carga y errores.
- Sincroniza la sesión con el perfil del empleado en la base de datos.
- Permite iniciar sesión, cerrar sesión y actualizar datos del usuario.
- Escucha cambios de autenticación en tiempo real.
- Expone un contexto **(`useAuth`)** para usar la autenticación en toda la app.

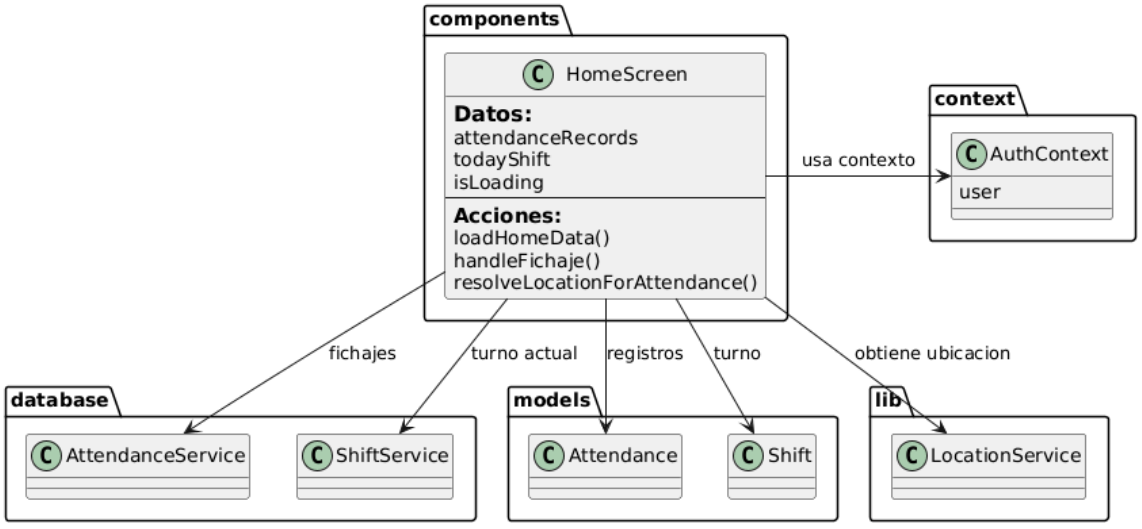


3. Capa de pantallas (src/screens/)

LoginScreen.js

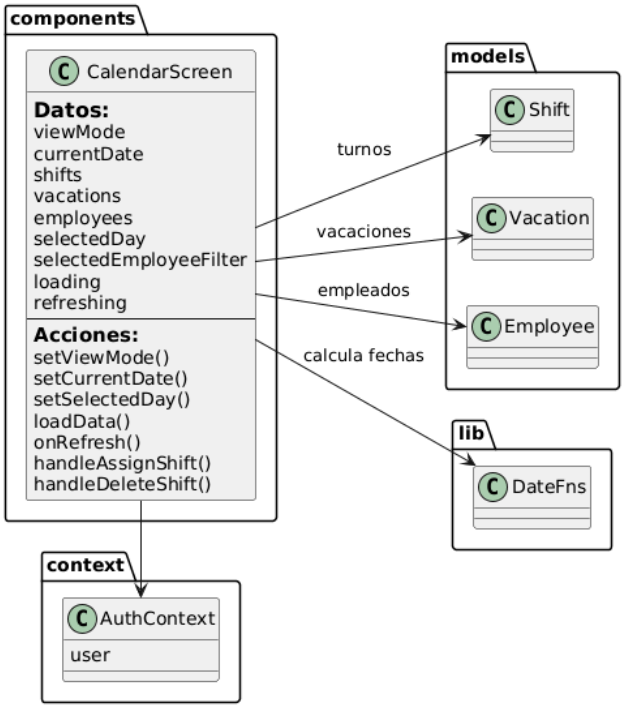


HomeScreen.js

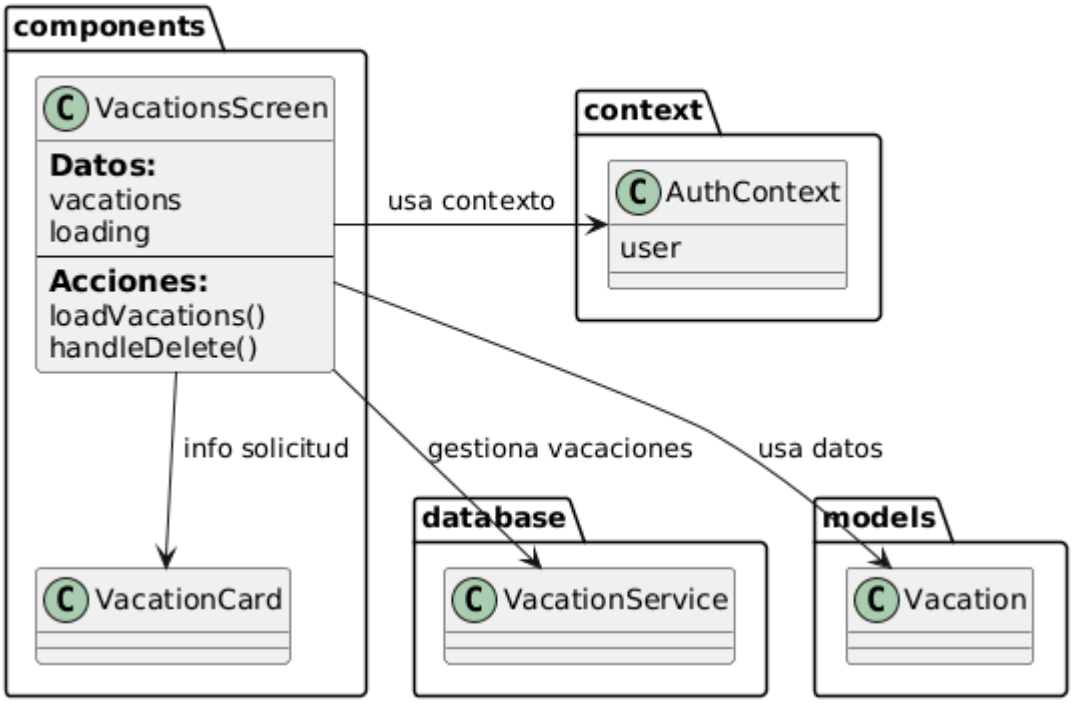


Rol empleado y administrador:

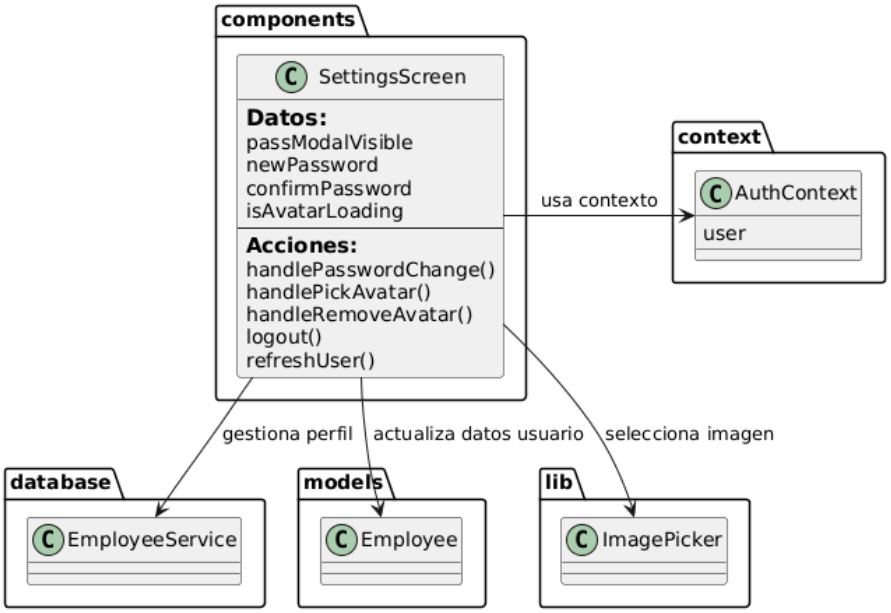
CalendarScreen.js



VacationScreen.js



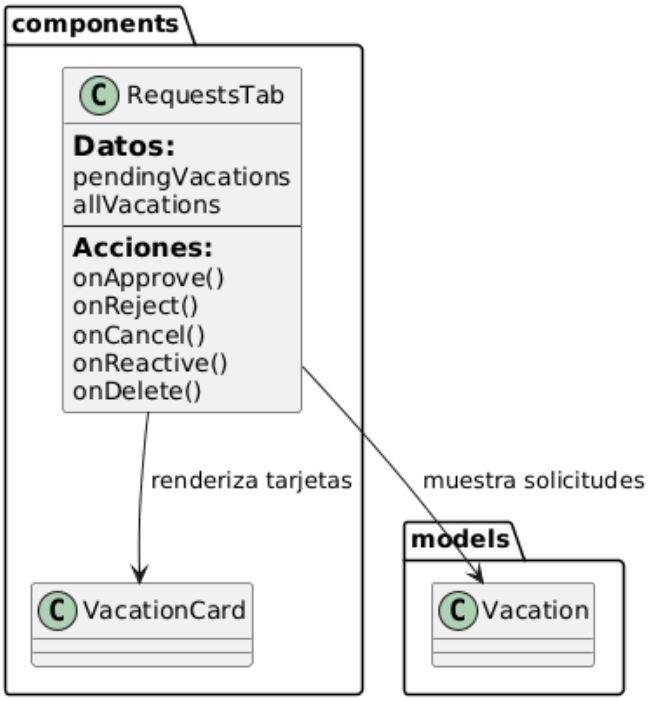
SettingsScreen.js



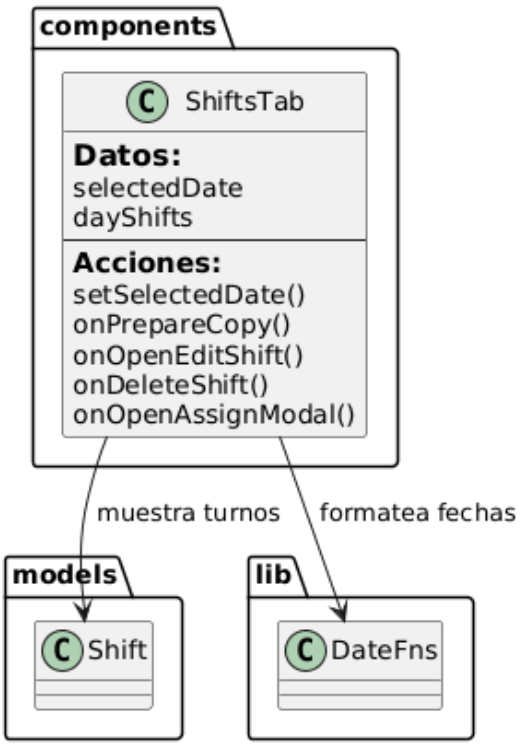
4. Capa de pantallas (src/screens/Admin)

Rol administrador (Panel de administración)

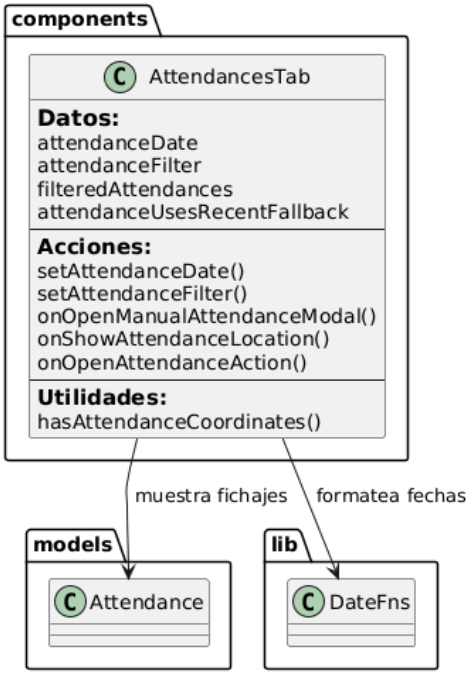
RequestTab.js



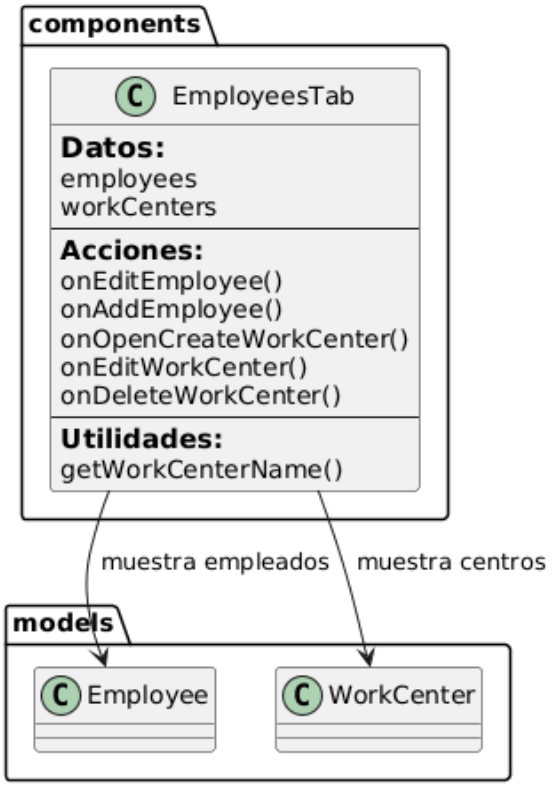
ShiftTab.js



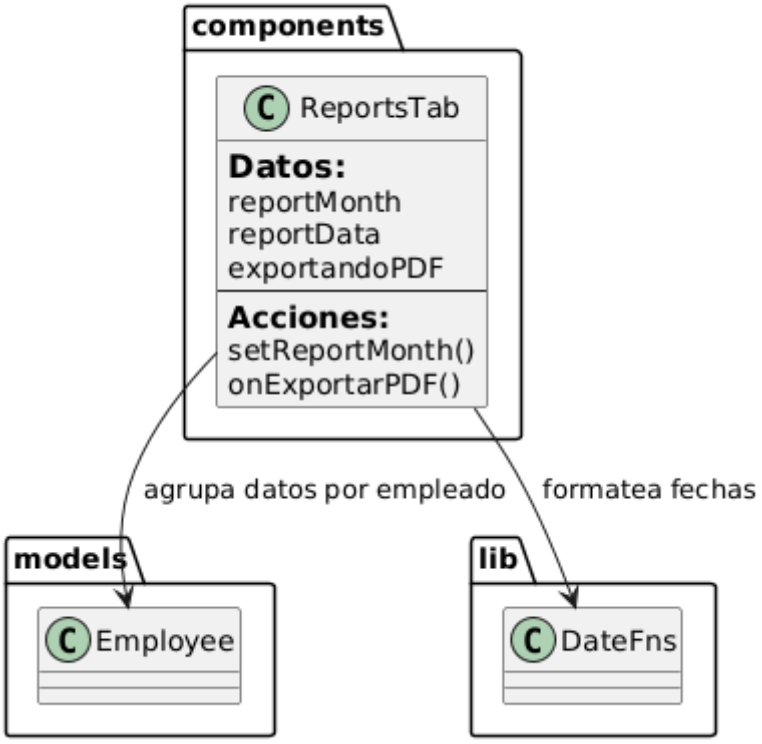
AttendanceTab.js



EmployeesTab.js



ReportsTab.js

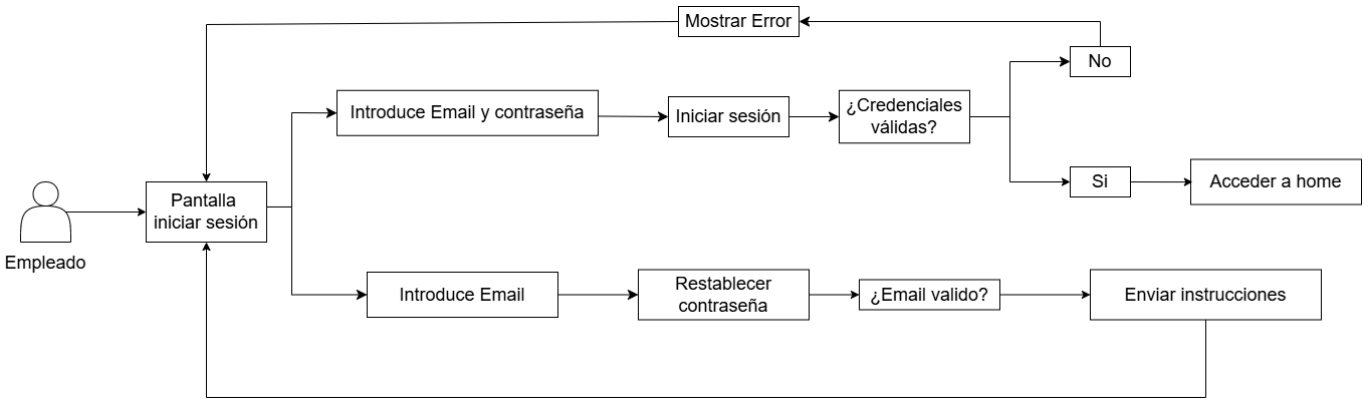


Anexo III – Casos de uso

Se presentan los principales casos de uso del sistema organizados por actor.

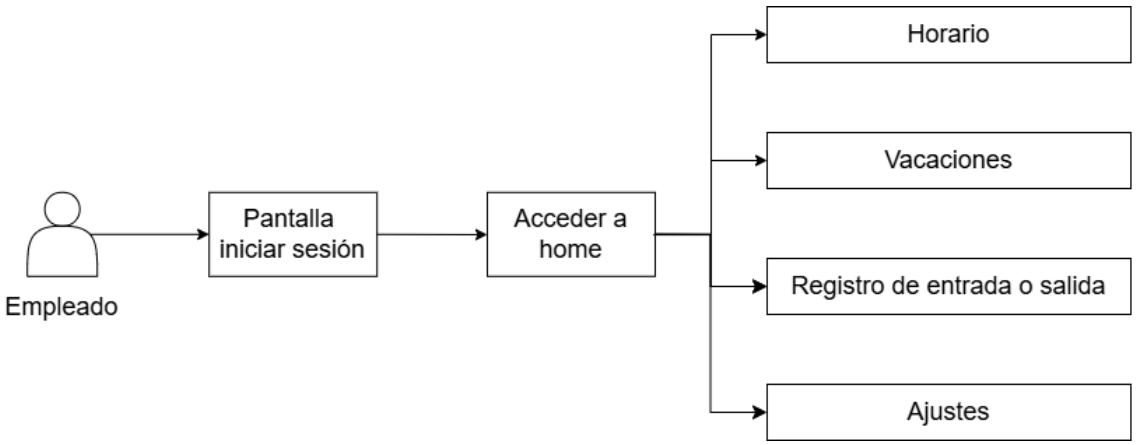
Iniciar sesión

Iniciar sesión y/o restablecer contraseña



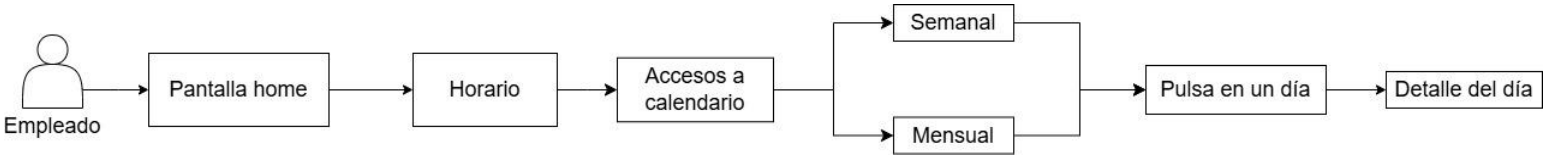
Actor: Empleado

Principales accesos que tiene el empleado



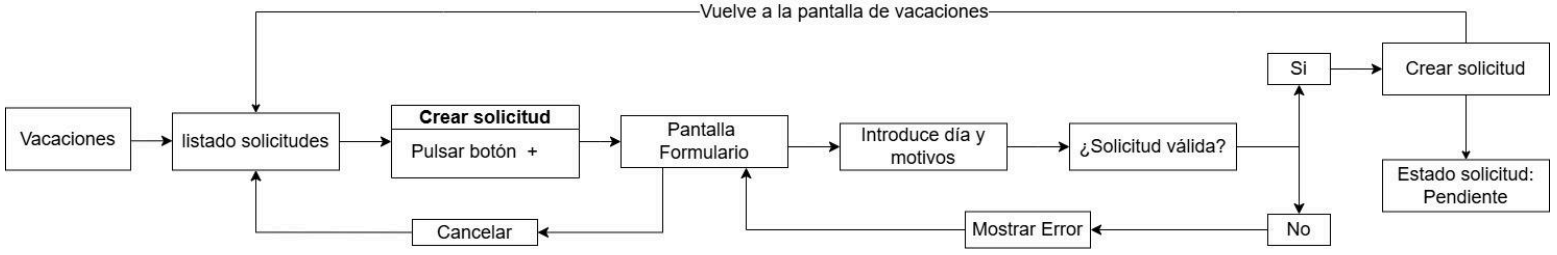
Horario

Visualización de turnos

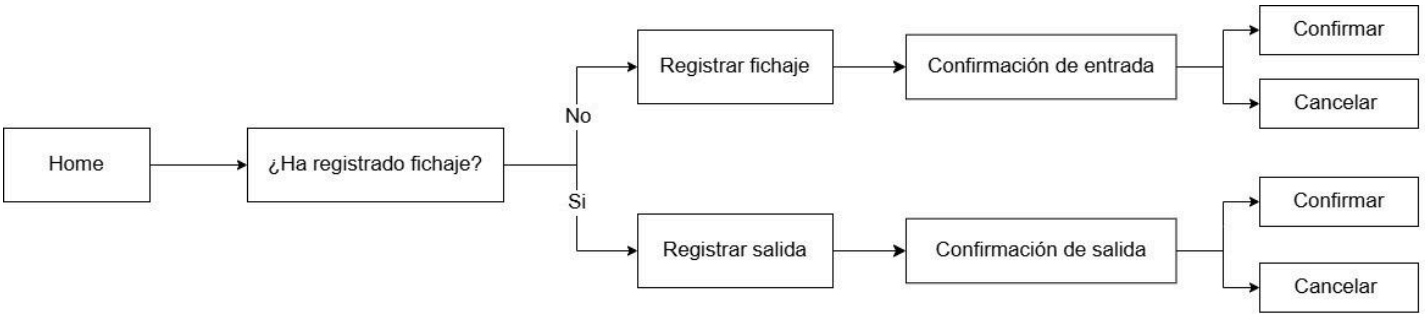


Vacaciones

Solicitar vacaciones



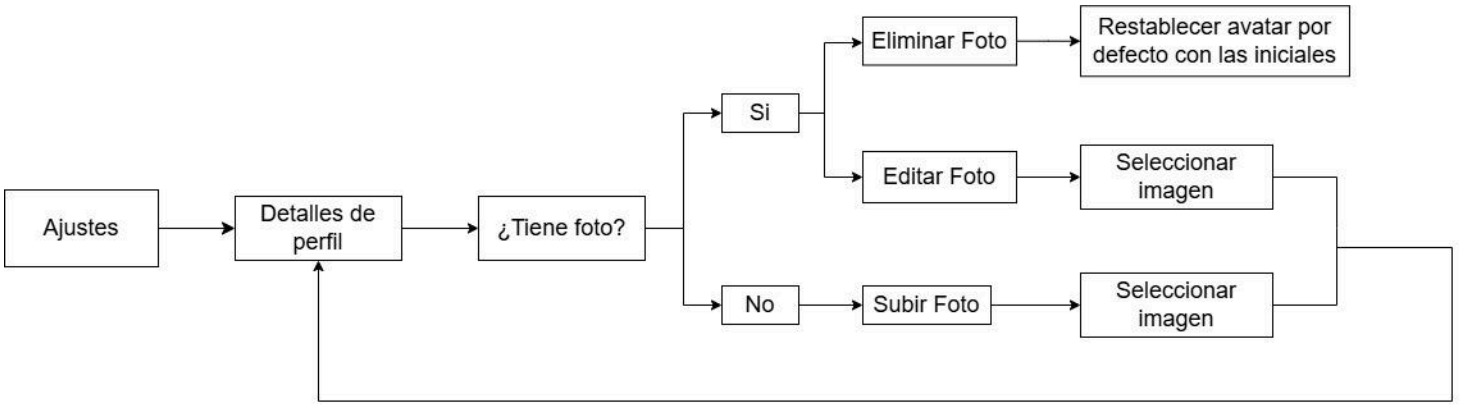
Registro de entrada



Ajustes

Configuración del perfil

Cambiar o eliminar foto

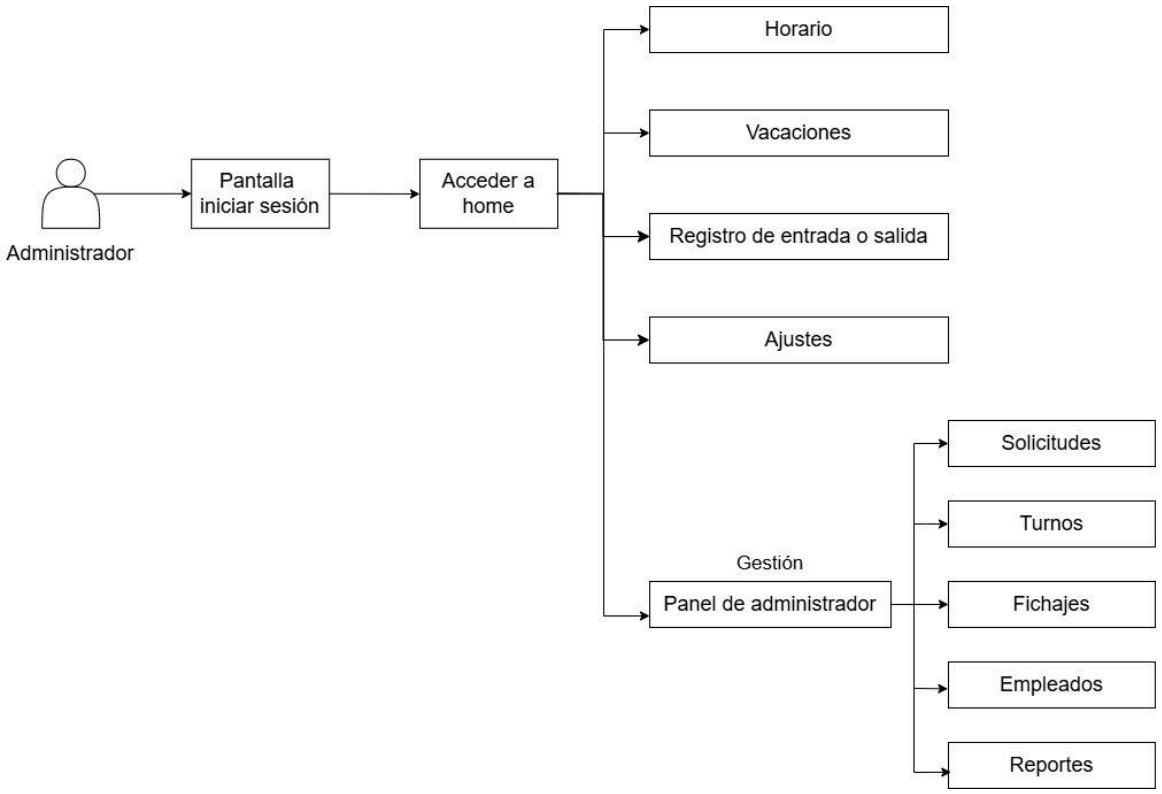


Cambiar contraseña



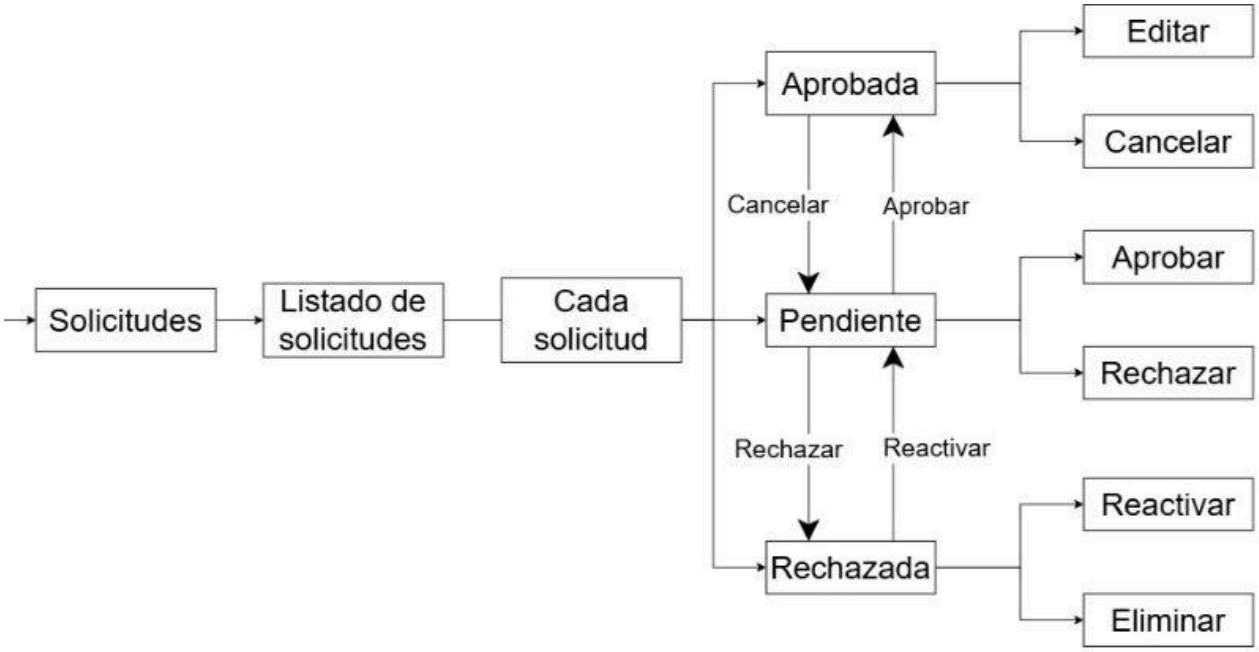
Actor: Administrador

Principales accesos que tiene el administrador



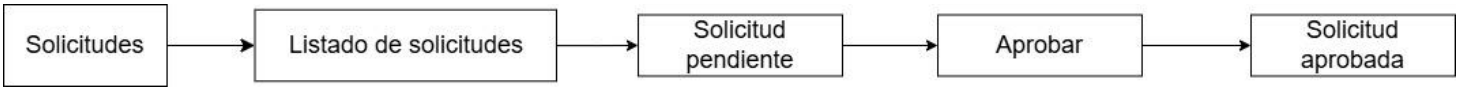
Solicitudes

Resumen del flujo de solicitudes

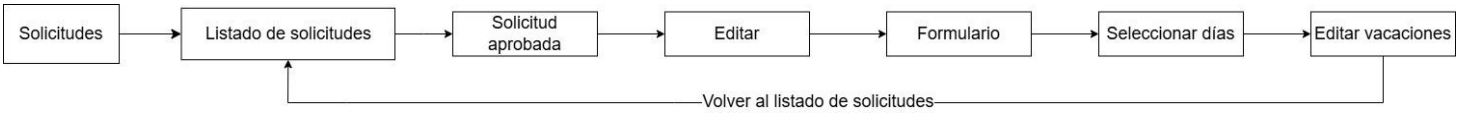


Casos de ejemplo:

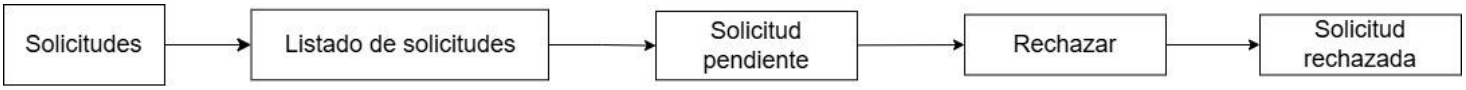
Aprobar solicitud



Editar solicitud aprobada

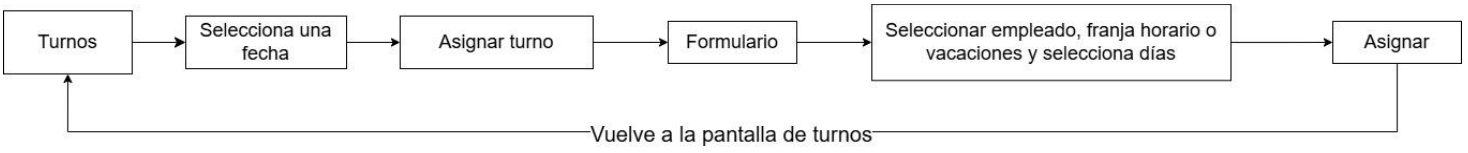


Rechazar solicitud pendiente

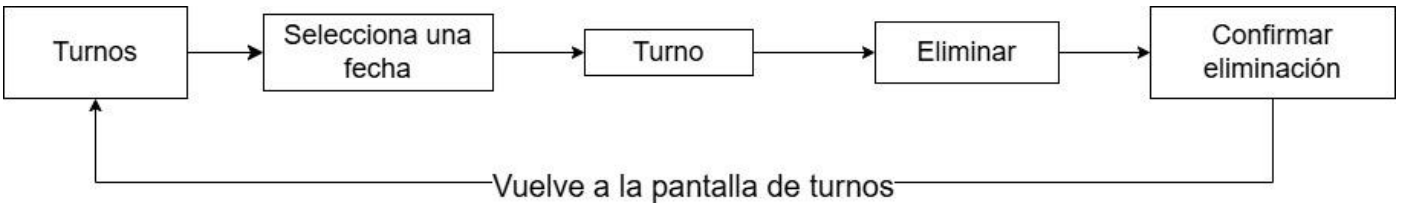


Turnos

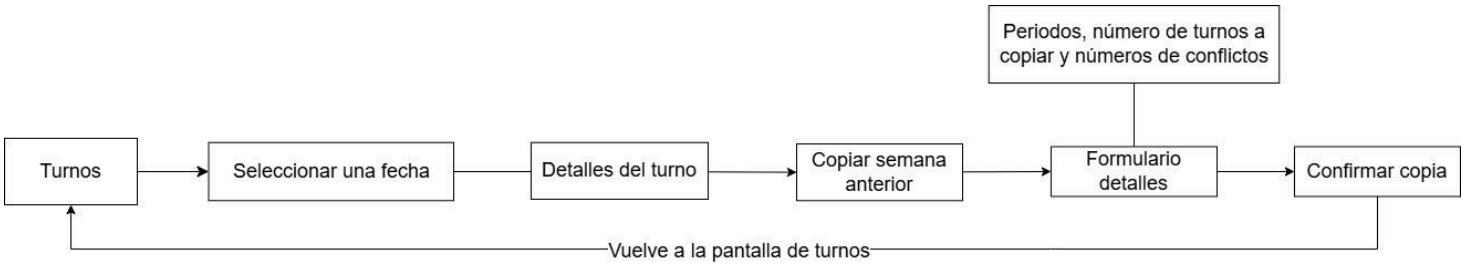
Asignar un turno



Eliminar un turno

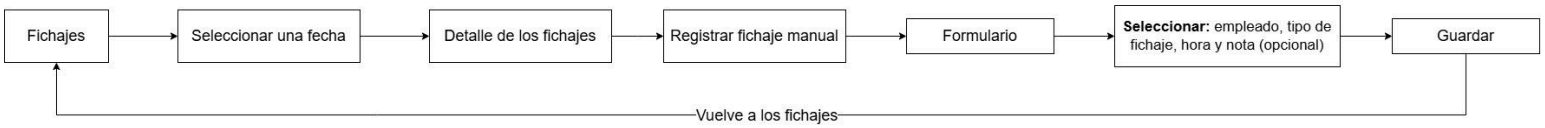


Copiar semana anterior

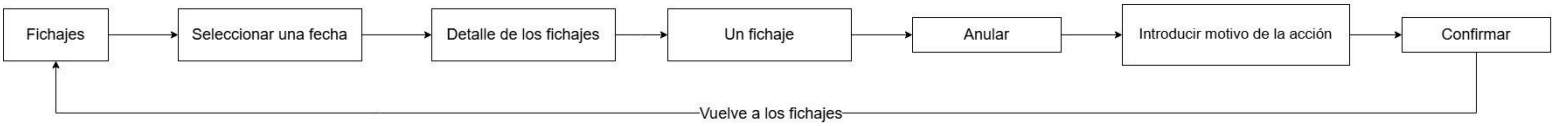


Fichajes

Registrar fichaje manual

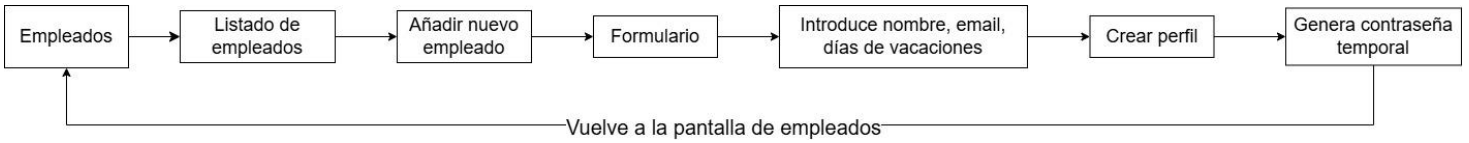


Anular fichaje

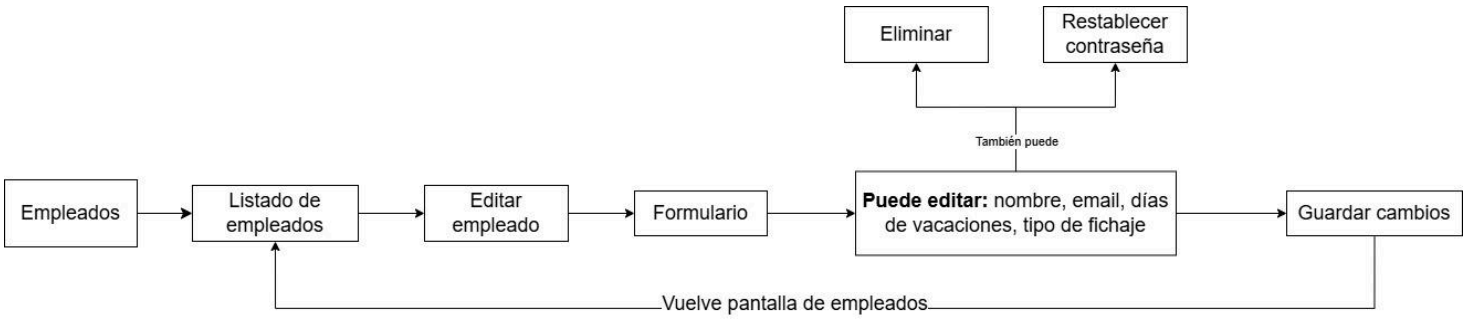


Empleados

Dar de alta un empleado (añadir nuevo empleado a la app)

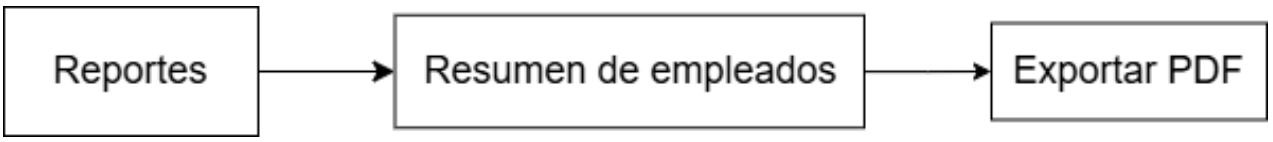


Editar empleado



Reportes

Exportar PDF



ID	Caso de uso	Descripción resumida
CU-07	Gestionar empleados	Da de alta, edita o elimina empleados del sistema.
CU-08	Asignar turno	Asigna un turno (Mañana/Tarde/Noche) a un empleado en una fecha, con validación de conflictos.
CU-09	Copiar turnos semana anterior	Clona los turnos de la semana anterior para un empleado como punto de partida.
CU-10	Eliminar turno	Elimina un turno asignado desde la vista diaria del calendario.
CU-11	Gestionar solicitudes de vacaciones	Aprueba o rechaza solicitudes de vacaciones pendientes.
CU-12	Consultar reportes mensuales	Obtiene el resumen de turnos y vacaciones de todos los empleados en un mes seleccionado.

Anexo IV – Capturas de pantalla de la aplicación

Las siguientes imágenes muestran las pantallas principales de TransferLog en funcionamiento.

Rol: Empleado

Inicio de sesión

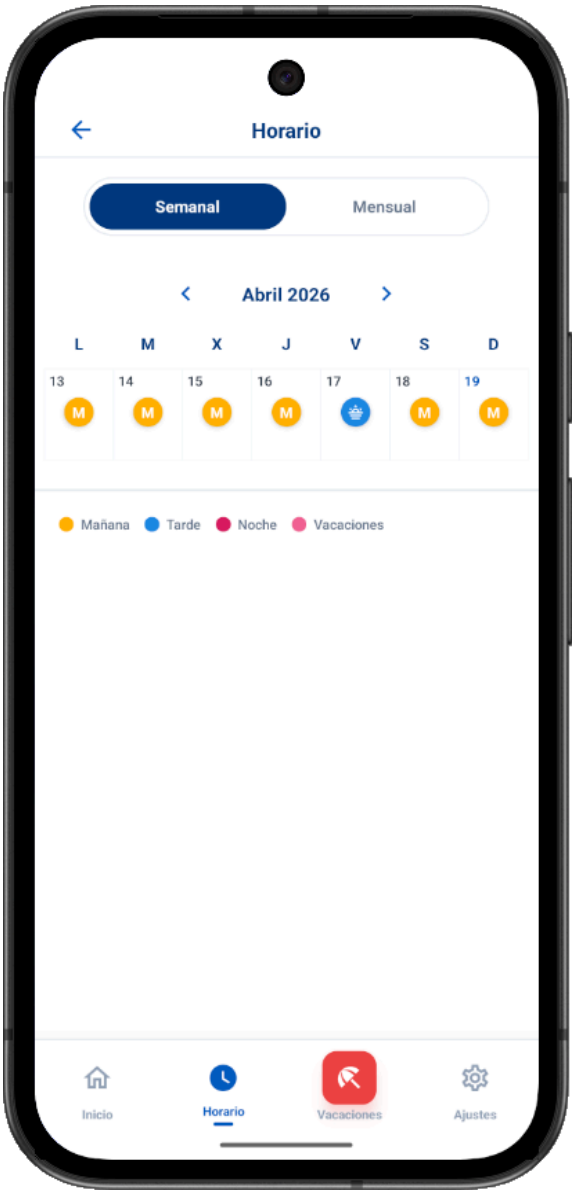


Inicio

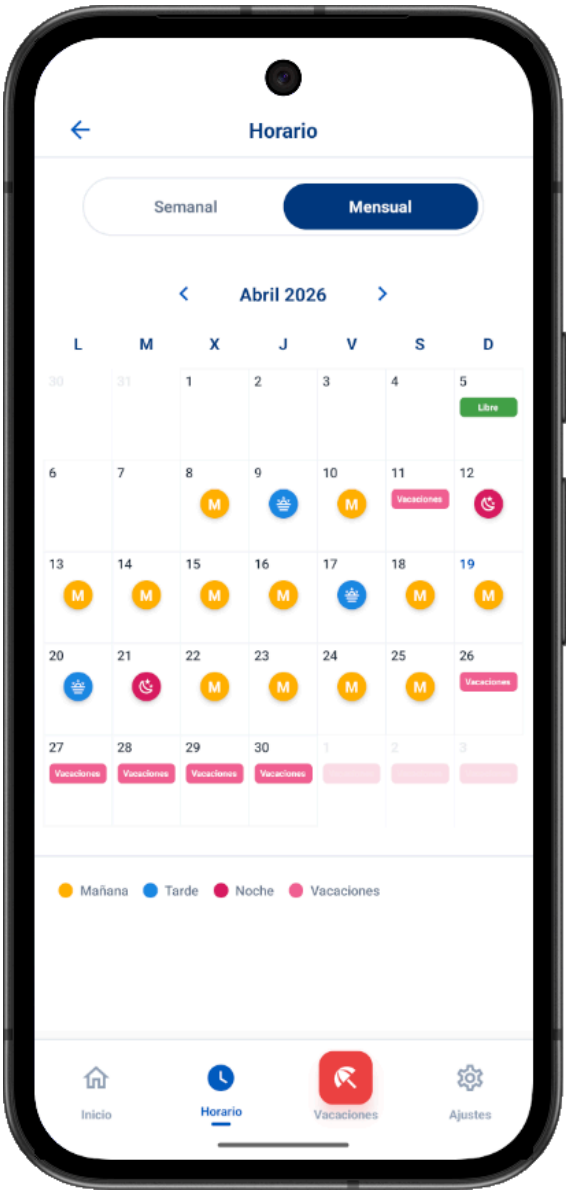


Horario

Semanal

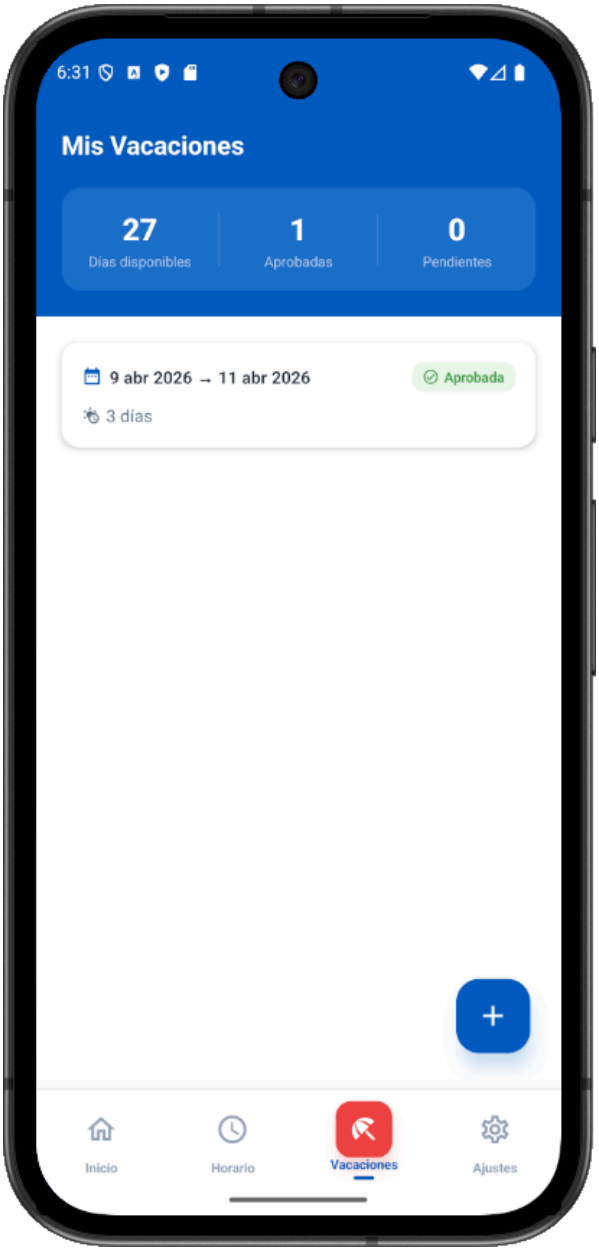


Mensual



Vacaciones

Principal

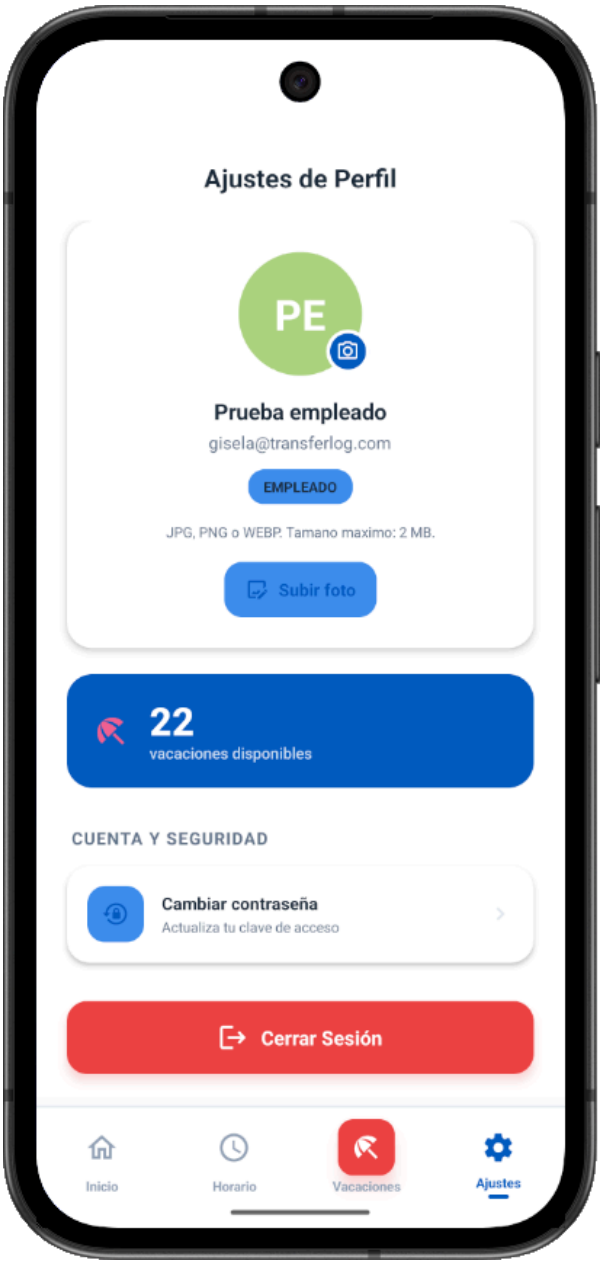


Solicitar Vacaciones

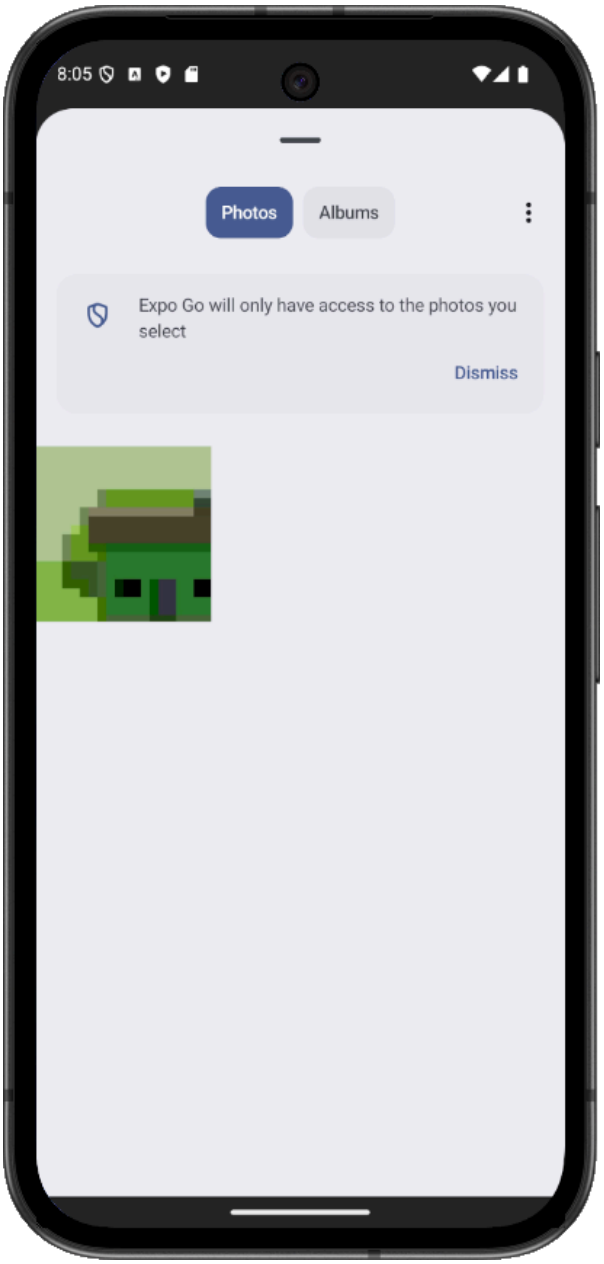


Ajustes

Detalles del perfil

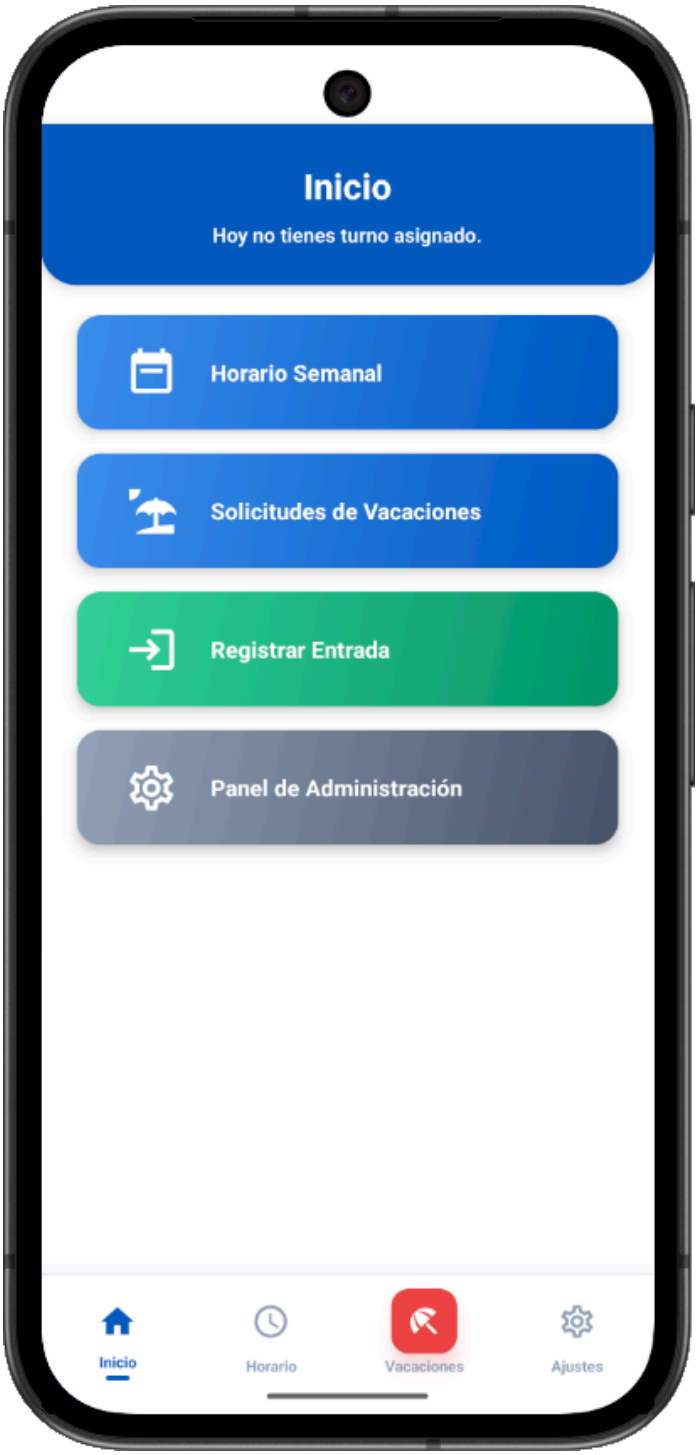


Cambiar foto



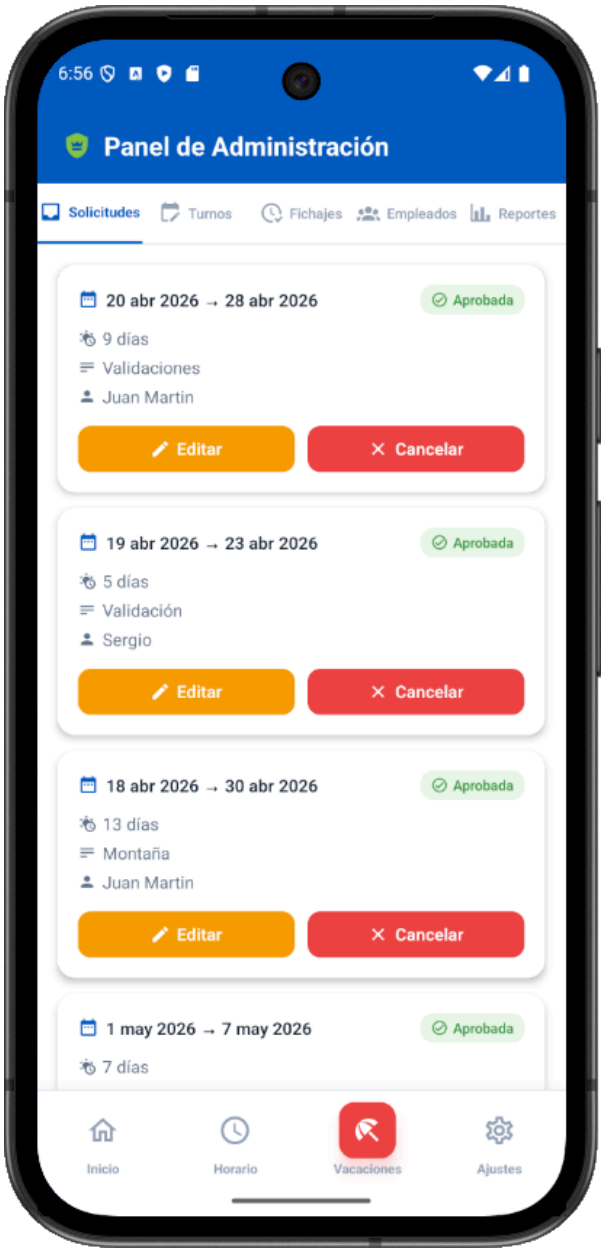
Rol: Administrador

Pantalla de inicio



Panel de administración

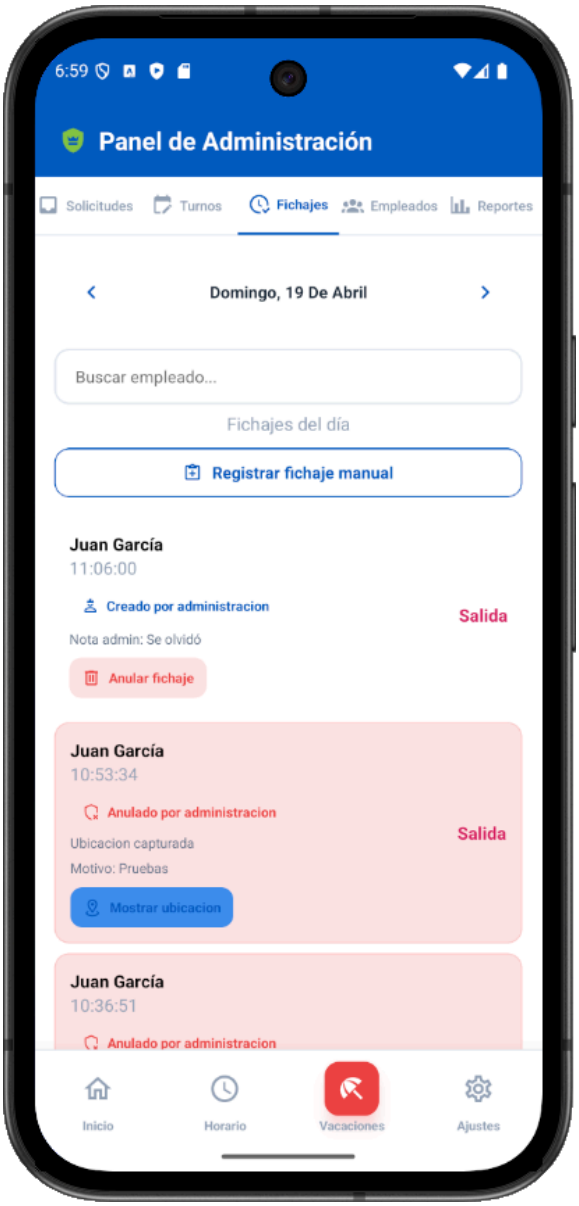
Solicitudes



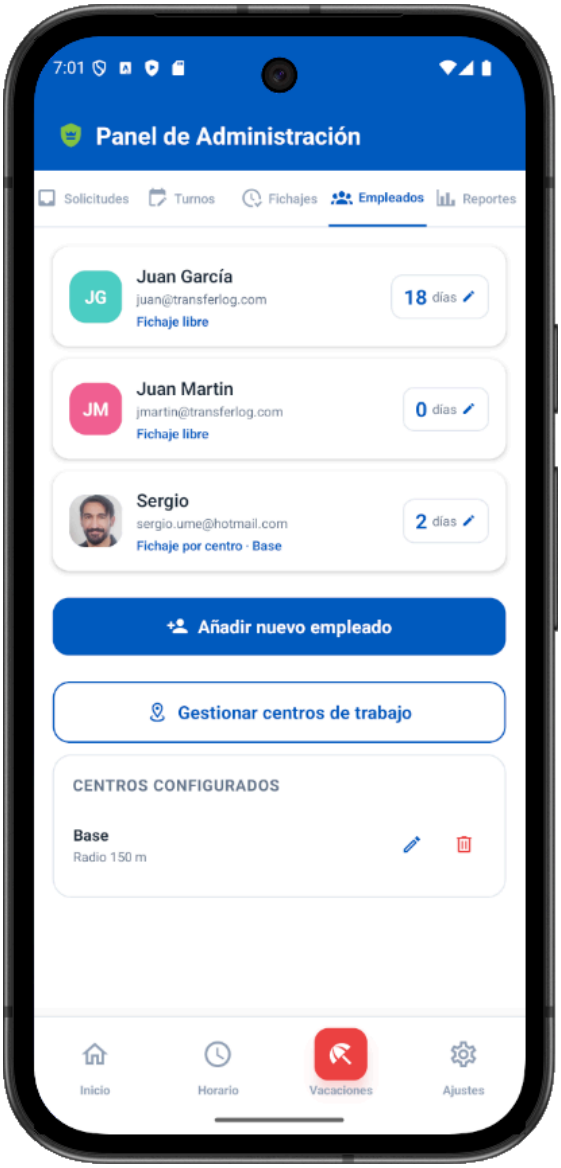
Turnos



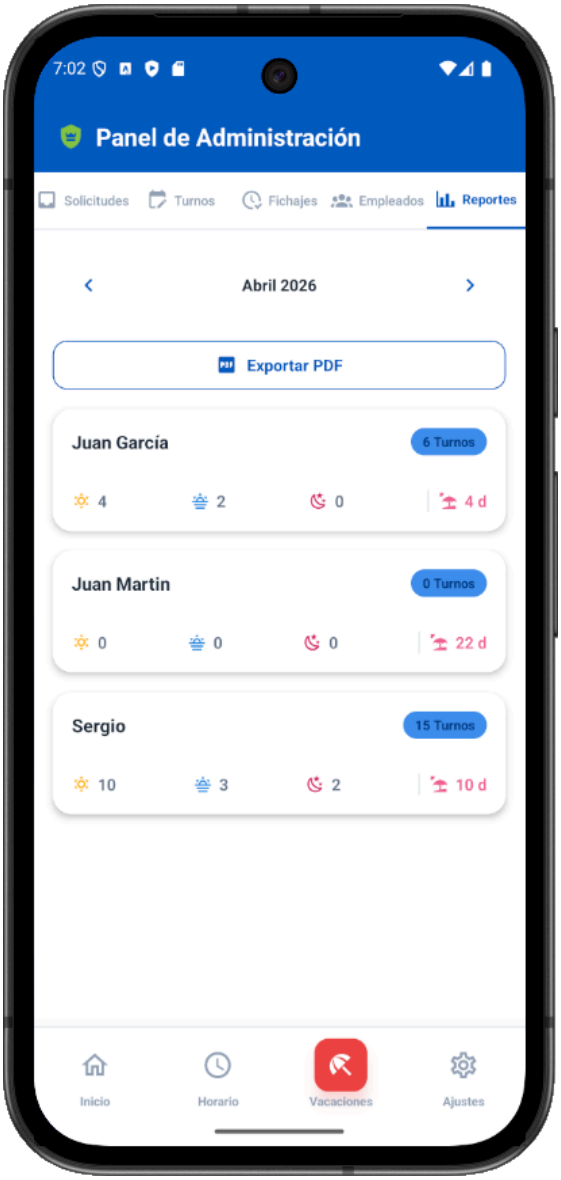
Fichajes


6:59
Panel de Administración
Solicitudes
Turnos
Fichajes
Empleados
Reportes
Domingo, 19 De Abril
Buscar empleado...
Fichajes del día
Registrar fichaje manual
Juan García
11:06:00
Creado por administracion
Salida
Nota admin: Se olvidó
Anular fichaje
Juan García
10:53:34
Anulado por administracion
Ubicacion capturada
Motivo: Pruebas
Salida
Mostrar ubicacion
Juan García
10:36:51
Anulado por administracion
Inicio
Horario
Vacaciones
Ajustes

Empleados



Reportes



Anexo V – Testing del sistema

Objetivo

El sistema de testing tiene como objetivo asegurar que la aplicación funcione correctamente, validando su lógica sin depender de servicios externos o bases de datos reales.

Su finalidad es comprobar que las reglas de negocio, las operaciones principales y las integraciones se comportan de manera consistente, garantizando la estabilidad del sistema y evitando regresiones ante cambios en el código.

Enfoque de testing

Se utilizan **tests unitarios con Jest**, con los siguientes principios:

- Mock completo de Supabase (supabase.from, supabase.rpc)
- Simulación de cadenas de query (fluent API chaining)
- Aislamiento total de base de datos real
- Validación de lógica de negocio, no de infraestructura

Mock de Supabase

Se simula la API de Supabase para controlar respuestas:

- .select()
- .eq()
- .order()
- .insert()
- .update()
- .delete()
- .rpc()

Esto permite testear el servicio sin dependencia de backend.

Archivos cubiertos:

- **vacationService.js** → Gestión completa de vacaciones (solicitudes, validaciones, estados y acciones como aprobar, rechazar o cancelar).

- **attendanceService.js** → Control de fichajes de empleados (registro, validaciones, geolocalización y gestión administrativa).

Casos de prueba implementados

Solicitud de vacaciones

Lectura de datos

- Historial de vacaciones por empleado
- Todas las vacaciones (con mapping de empleados)
- Vacaciones pendientes
- Próximas vacaciones (limitadas a 3)

Lógica de negocio

- Cálculo de días entre fechas
- Validación de días disponibles
- ❌ Error si no hay saldo suficiente
- ✅ Inserción correcta si la validación es exit osa

Operaciones transaccionales (RPC)

- approve_vacation_transactional
- edit_vacation_transactional
- cancel_vacation_transactional

Incluye:

- Invocación correcta de RPC
- Manejo de errores

Gestión de estados

- **rejectVacation()** → rejected
- **reactiveVacation()** → pending
- **deleteVacation()** → eliminación

Validaciones cubiertas

- Días suficientes obligatorios
- Manejo de errores de Supabase

Gestión de fichajes

Registro de fichajes

- Evita duplicidad de entradas y salidas en el mismo día
- Inserta correctamente cuando no hay conflictos.

Registro con geolocalización

- Requiere empleado autenticado.
- Soporta políticas:
  - manual_only
  - anywhere
  - assigned_center
- Valida:
  - Centro asignado
  - Existencia del centro
  - Ubicación del usuario
- Controla:
  - Distancia al centro
  - Precisión GPS
- Bloquea fichajes fuera del radio permitido.
- Inserta:
  - Fichaje validado en centro
  - Fichaje con ubicación opcional (si aplica política `anywhere`)

Fichaje manual por administrador

- Requiere administrador y empleado
- Valida:
  - Tipo de fichaje (in / out)
  - Formato de fecha válido
- Evita:
  - Duplicados de entrada/salida
  - Inconsistencias de secuencia (salida sin entrada previa)
- Detecta migraciones faltantes del sistema
- Inserta con auditoría:
  - entry_mode
  - created_by_employee_id
  - admin_note

Anulación de fichajes

- Requiere motivo y administrador
- Evita anular fichajes ya anulados
- Solo permite anular el último fichaje activo del día
- Maneja errores y migraciones faltantes
- Actualiza estado a voided con auditoría:
    - voided_by_employee_id
    - void_reason

Consultas

- **getRecentAttendances**
    - Limita resultados
    - Normaliza datos (estado y modo de entrada por defecto)
- **getAllAttendancesByDate**
    - Recupera fichajes por fecha
    - Compatible con estructuras antiguas (sin columnas de auditoría)

Anexo VI – Instrucciones de instalación y ejecución

Requisitos previos

- Node.js 18 LTS o superior.
- npm 9 o superior.
- Expo CLI: npm install -g expo-cli
- Expo Go instalado en el dispositivo móvil de prueba (disponible en App Store / Google Play).
- Cuenta en Supabase con el proyecto configurado y las credenciales de API disponibles.

Pasos de instalación

1. Clonar el repositorio del proyecto.
2. Ejecutar npm install en la raíz del proyecto para instalar las dependencias.
3. Configurar las variables de entorno de Supabase en src/lib/supabase.js (URL del proyecto y clave pública anon key).
4. Ejecutar npx expo start para lanzar el servidor de desarrollo de Expo.
5. Escanear el código QR con la app Expo Go desde el dispositivo móvil, o ejecutar en emulador.

Credenciales de prueba

Rol	Email de prueba	Contraseña
Administrador	admin@transferlog.com	Ver documentación interna
Empleado	empleado@transferlog.com	Ver documentación interna

— Fin de la Memoria de Proyecto —